

Projektni zadatak 3
VVS

Tim:	Brojevi indeksa:
Ismail Džafić	19588
Tarik Lolić	19513
Eldar Hadžiselimović	19434
Faruk Avdagić	19350

Sadržaj

1 Zadatak 1 – Blackbox Testiranje **4**

1.1 Testno Okruženje . . . 4

1.2 Testni Slučajevi (Test Cases) . . . 4

1.2.1 TC-01: Prijava sa validnim korisničkim podacima . . . 4

1.2.2 TC-02: Prijava sa zaključanim korisnikom . . . 4

1.2.3 TC-03: Validacija obaveznih polja (Checkout) . . . 5

1.2.4 TC-04: Sortiranje proizvoda po cijeni . . . 6

1.2.5 TC-05: End-to-End kupovina . . . 7

1.3 Prijavljeni Bagovi (Bug Reports) . . . 7

1.3.1 BUG-01: Neispravne slike proizvoda . . . 7

1.3.2 BUG-02: Greška pri unosu prezimena . . . 7

1.4 Zaključak . . . 10

2 Zadatak 2 – Azure DevOps **10**

2.1 Automatizacija pomoću Selenium IDE alata . . . 10

2.1.1 Detaljni pregled implementiranih scenarija . . . 11

2.2 NUnit Selenium Framework . . . 14

2.2.1 Arhitektura projekta (Design Patterns) . . . 14

2.2.2 Izvršavanje i rezultati testiranja . . . 17

3 Zadatak 3 – Postman API Testiranje **19**

3.1 Konfiguracija okruženja . . . 19

3.2 Pokretanje kolekcije testova . . . 20

3.3 Store – Upravljanje narudžbama . . . 20

3.3.1 Get Inventory . . . 20

3.3.2 POST /store/order – Kreiranje narudžbe . . . 21

3.3.3 GET /store/order/{orderId} – Dohvatanje narudžbe . . . 25

3.3.4 DELETE /store/order/{orderId} – Brisanje narudžbe . . . 29

3.4 Pet – Upravljanje ljubimcima . . . 32

3.4.1 POST /pet – Kreiranje ljubimca . . . 32

3.4.2 GET /pet/{petId} – Dohvatanje ljubimca . . . 36

3.4.3 GET /pet/findByStatus – Pretraga ljubimaca po statusu . . . 39

3.4.4 GET /pet/findByTags – Pretraga po tagovima (DEPRECATED) . . . 41

3.4.5 PUT /pet – Azuriranje ljubimca . . . 41

3.4.6 POST /pet/{petId} – Ažuriranje putem forme . . . 42

3.4.7 POST /pet/{petId}/uploadImage – Upload slike . . . 44

3.4.8 DELETE /pet/{petId} – Brisanje ljubimca . . . 44

3.5 User – Upravljanje korisnicima . . . 47

3.5.1 POST /user – Kreiranje korisnika . . . 47

3.5.2 POST /user/createWithArray – Bulk kreiranje . . . 51

3.5.3 POST /user/createWithList – Bulk kreiranje . . . 52

3.5.4 GET /user/login – Prijava korisnika . . . 53

3.5.5 GET /user/{username} – Dohvatanje korisnika . . . 55

3.5.6 PUT /user/{username} – Ažuriranje korisnika . . . 58

3.5.7 GET /user/logout – Odjava korisnika . . . 60

3.5.8 DELETE /user/{username} – Brisanje korisnika . . . 61
3.5.9 Cleanup zahtjevi – Brisanje test korisnika . . . 62
3.6 Metodologija i napomene o testiranju . . . 64
3.6.1 Poznati problemi . . . 64
3.6.2 Pre-request skripte . . . 64
3.6.3 Strategija robustnosti . . . 64
3.7 Sumarni pregled testova . . . 64
3.8 Tipovi implementiranih testova . . . 64
3.9 Rezultati Newman testiranja (CLI) . . . 65
3.9.1 Sumar rezultata . . . 65
3.9.2 Statistika izvršavanja . . . 65
3.10 Rezultati po sekcijama . . . 65
3.10.1 Store sekcija (18 zahtjeva) . . . 65
3.10.2 Pet sekcija (21 zahtjev) . . . 65
3.10.3 User sekcija (24 zahtjeva) . . . 66
3.11 Sveobuhvatna analiza . . . 66
3.12 Zaključak . . . 68

4 Zadatak 4 – Load Testiranje **68**
4.1 Alat k6 - Load Testiranje . . . 68
4.1.1 Metodologija testiranja . . . 68
4.1.2 Zbirni rezultati . . . 69
4.1.3 Objašnjenje ključnih metrika . . . 69
4.1.4 Analiza po grupama operacija . . . 69
4.1.5 Historijski trendovi performansi . . . 71
4.1.6 Zaključak k6 analize . . . 73
4.2 Apache JMeter - Load i Stress testiranje . . . 74
4.2.1 Konfiguracija Test Plana . . . 74
4.2.2 Analiza Rezultata . . . 74
4.2.3 Vizualni Pregled Rezultata . . . 77
4.2.4 Vremenske Analize - Over Time Grafici . . . 79
4.2.5 Throughput i Skalabilnost - Analiza po Zahtjevu . . . 82
4.2.6 Response Time Distribution - Analiza Raspodjele Latencija . . . 84
4.2.7 Zaključak . . . 87

1 Zadatak 1 – Blackbox Testiranje

Ovo poglavlje sadrži izvještaj o provedenom funkcionalnom Black Box testiranju web aplikacije **Swag Labs (SauceDemo)**. Cilj testiranja bio je verifikacija ključnih funkcionalnosti aplikacije (Login, Inventory, Checkout) primjenom tehnika opisanih u materijalima "Predavanje 9"

Korištene tehnike dizajniranja testova:

- **Ekvivalentno particionisanje (EP):** Testiranje validnih i nevalidnih klasa unosa.
- **Analiza graničnih vrijednosti (BVA):** Testiranje praznih (null) vrijednosti.
- **Use Case Testiranje:** Testiranje end-to-end scenarija.

1.1 Testno Okruženje

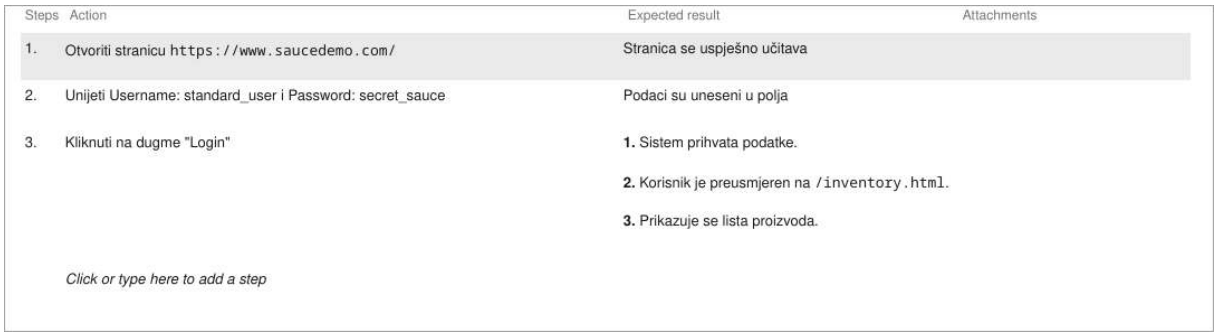
- **URL Aplikacije:** https://www.saucedemo.com/
- **Browser:** Google Chrome
- **OS:** Windows 11

1.2 Testni Slučajevi (Test Cases)

U nastavku je prikazano 5 kreiranih testnih slučajeva formatiranih za unos u Azure DevOps.

1.2.1 TC-01: Prijava sa validnim korisničkim podacima

ID	TC-01
Naslov	Prijava sa validnim korisničkim podacima
Tehnika	Ekvivalentno particionisanje (Validna klasa)
Prioritet	1
Opis	Ovaj testni slučaj verifikuje osnovnu funkcionalnost autentifikacije korisnika primjenom tehnike Ekvivalentnog particionisanja (Validna klasa). Cilj je potvrditi da sistem ispravno dozvoljava pristup registrovanom korisniku (standard_user) i uspješno ga preusmjerava na internu stranicu aplikacije (Inventory). Test potvrđuje "Happy Path" za Login funkcionalnost.
Link	Login stranica

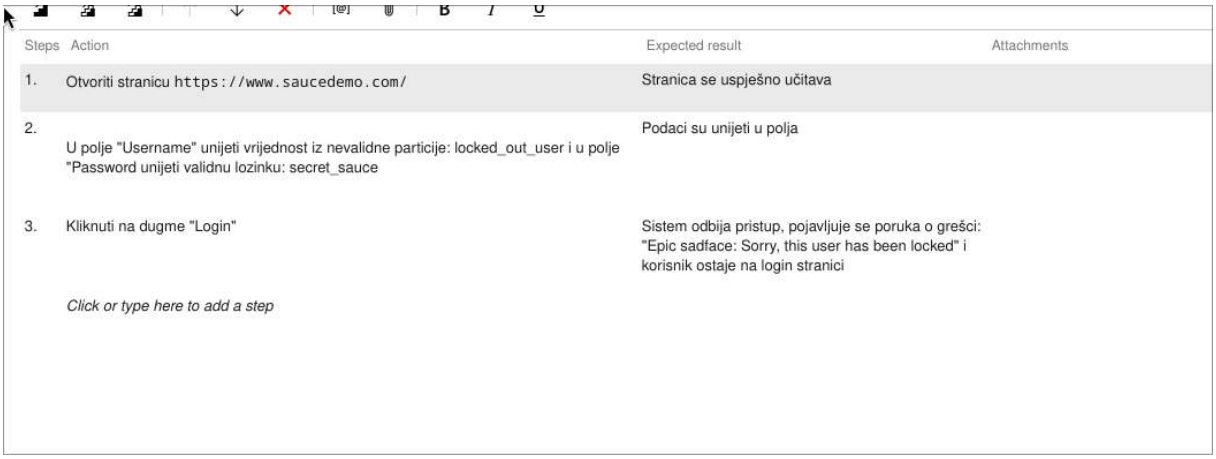


Steps	Action	Expected result	Attachments
1.	Otvoriti stranicu `https://www.saucedemo.com/`	Stranica se uspješno učitava	
2.	Unijeti Username: standard_user i Password: secret_sauce	Podaci su uneseni u polja	
3.	Kliknuti na dugme "Login"	**1.** Sistem prihvata podatke. **2.** Korisnik je preusmjeren na `/inventory.html`. **3.** Prikazuje se lista proizvoda.	
	Click or type here to add a step		

Slika 1: Prikaz svih koraka testnog slučaja 1 i očekivanih rezultata svakog koraka

1.2.2 TC-02: Prijava sa zaključanim korisnikom

ID	TC-02
Naslov	Validacija prijave sa zaključanim korisnikom
Tehnika	Ekvivalentno particionisanje (Nevalidna klasa)
Prioritet	2
Opis	Ovaj testni slučaj provjerava mehanizam upravljanja greškama (Error Handling) i sigurnosne aspekte prijave primjenom tehnike Ekvivalentnog particionisanja (Nevalidna klasa). Cilj je potvrditi da sistem ispravno identifikuje status korisnika (locked_out_user), blokira pristup aplikaciji i prikazuje odgovarajuću, jasnu poruku o grešci korisniku, sprječavajući neovlašten ili nevalidan pristup.
Link	Login stranica



Steps	Action	Expected result	Attachments
1.	Otvoriti stranicu `https://www.saucedemo.com/`	Stranica se uspješno učitava	
2.	U polje "Username" unijeti vrijednost iz nevalidne particije: locked_out_user i u polje "Password unijeti validnu lozinku: secret_sauce	Podaci su unijeti u polja	
3.	Kliknuti na dugme "Login"	Sistem odbija pristup, pojavljuje se poruka o grešci: "Epic sadface: Sorry, this user has been locked" i korisnik ostaje na login stranici	
	Click or type here to add a step		

Slika 2: Prikaz svih koraka testnog slučaja 2 i očekivanih rezultata svakog koraka

1.2.3 TC-03: Validacija obaveznih polja (Checkout)

ID	TC-03
Naslov	Validacija polja za poštanski broj (Zip Code) - Prazan unos
Tehnika	Analiza graničnih vrijednosti (Null/Empty value)
Prioritet	2
Opis	Ovaj test se fokusira na validaciju unosa na formi za naplatu (Checkout) primjenom tehnike Analize graničnih vrijednosti (BVA), specifično testirajući praznu (null) vrijednost. Cilj je osigurati da obavezno polje "Zip/Postal Code" ne prihvata prazan unos i da sistem blokira prelazak na sljedeći korak (Overview) dok se ne unesu validni podaci, čime se osigurava integritet podataka narudžbe.
Link	Stranica za checkout (NAPOMENA: prijaviti se prethodno kao standard_user i dodati artikle u košaricu)

Steps	Action	Expected result	Attachments
1.	Otvoriti stranicu `https://www.saucedemo.com/`	Uspješno se učitava stranica	
2.	Unijeti u *standard_user* u polje "Username" i *secret_sauce* u polje "Password"	Podaci su uneseni u sva polja	
3.	Kliknuti na dugme "Login"	Uspješno se učitava stranica inventory.html	
4.	Dodati artikal "Sauce Labs Bike Light" u košaricu	Ažurirana ikonica košarice u gornjem desnom uglu	
5.	Kliknuti na ikonicu košarice	Uspješno se učitava stranica cart.html	
6.	Kliknuti na dugme "Checkout"	Uspješno se učitava stranica checkout_step_one.html	
7.	Unijeti validno "First name" (npr. "Test") i validno "Last name" (npr. "User")	Uneseni su podaci u poljima "First name" i "Last name"	
8.	Ostaviti polje "Zip/Postal Code" praznim (Null vrijednost)	Nije unesen podatak u polju "Zip/Postal Code"	
9.	Kliknuti na dugme "Continue"	Sistem sprječava prelazak na sljedeći korak i prikazuje se validacijska poruka: "Error: Postal Code is required"	
	Click or type here to add a step		

Slika 3: Prikaz svih koraka testnog slučaja 3 i očekivanih rezultata svakog koraka

1.2.4 TC-04: Sortiranje proizvoda po cijeni

ID	TC-04
Naslov	Sortiranje proizvoda po cijeni (Low to High)
Tehnika	Funkcionalno testiranje
Prioritet	2
Opis	Ovaj test verifikuje ispravnost algoritma za sortiranje na stranici proizvoda (Inventory). Cilj je potvrditi da funkcionalnost filtriranja ispravno obrađuje zahtjev korisnika za promjenom redoslijeda prikaza. Test provjerava da li odabir opcije "Price (low to high)" vizuelno rearanžira proizvode tako da je prvi prikazani artikal onaj sa najnižom cijenom, a posljednji onaj sa najvišom, osiguravajući tačnost prikazanih informacija.
Link	Početna stranica (NAPOMENA: prijaviti se prvo kao standard_user)

Steps	Action	Expected result	Attachments
1.	Otvoriti stranicu `https://www.saucedemo.com/`	Uspješno se učitava stranica	
2.	Upisati standard_user u polje "Username" i secret_sauce u polje "Password"	Podaci su upisani u sva polja	
3.	Kliknuti na dugme "Login"	Uspješno se učitava stranica inventory.html	
4.	Otvoriti padajući meni za sortiranje i odabrati opciju "Price (low to high)"	Stranica automatski sortira listu artikala od najjeftinijeg do najskupljeg	
5.	Provjeriti cijenu prvog artikla na listi i cijenu posljednjeg artikla na listi	Prvi artikal ima najnižu cijenu ($7.99), a zadnji artikal ima najvišu cijenu ($49.99)	
	Click or type here to add a step		

Slika 4: Prikaz svih koraka testnog slučaja 4 i očekivanih rezultata svakog koraka

1.2.5 TC-05: End-to-End kupovina

ID	TC-05
Naslov	Uspješna kupovina proizvoda (Happy Path)
Tehnika	Use Case Testing
Prioritet	1
Opis	Ovaj testni slučaj predstavlja Use Case (Scenario) testiranje koje pokriva kompletan životni ciklus narudžbe ("Happy Path"). Cilj je verifikovati integraciju različitih modula aplikacije (Inventory -> Cart -> Checkout -> Overview -> Finish). Test potvrđuje da korisnik može uspješno proći kroz sve korake kupovine bez grešaka i dobiti potvrdnu poruku o uspješnoj narudžbi.
Link	Stranica za pregled korpe (NAPOMENA: prijaviti se prvo kao standard_user i odraditi sve navedene korake)

Steps	Action	Expected result	Attachments
1.	Otvoriti stranicu `https://www.saucedemo.com/`	Uspješno se učitava stranica	
2.	Unijeti u polje Username: standard_user i u polje Password: secret_sauce	Podaci su popunjeni u svim poljima	
3.	Kliknuti na dugme "Login"	Uspješno se učitava stranica inventory.html	
4.	Kliknuti "Add to cart" na proizvodu "Sauce Labs Backpack"	Ažurira se ikonica košarice	
5.	Kliknuti na ikonu košarice	Uspješno se učitava stranica cart.html	
6.	Kliknuti dugme "Checkout"	Uspješno se učitava stranica checkout_step_one.html	
7.	Popuniti sva polja (First Name, Last Name, Zip/Postal Code) validnim podacima	Podaci su popunjeni u svim poljima	
8.	Kliknuti na dugme "Continue"	Uspješno se učitava stranica checkout_step_two.html	
9.	Kliknuti na dugme "Finish"	Uspješno se učitava stranica checkout_complete.html sa vidljivom porukom: "Thank you for your order!"	
10.	Kliknuti na dugme "Back Home"	Uspješno se učitava stranica inventory.html	

Slika 5: Prikaz svih koraka testnog slučaja 5 i očekivanih rezultata svakog koraka

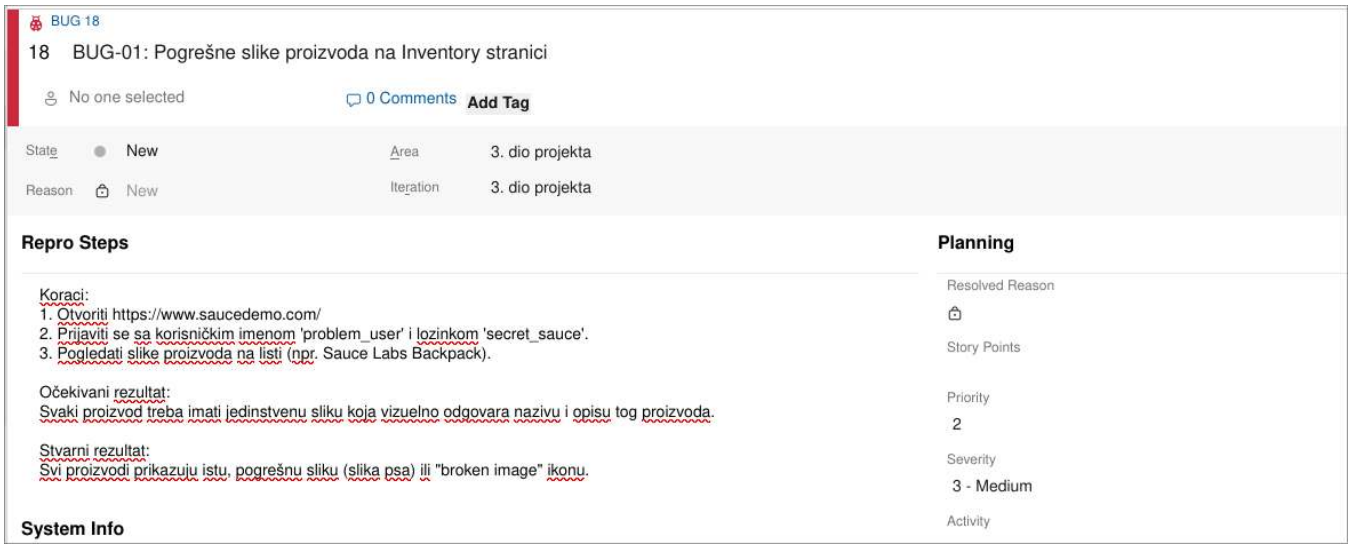
1.3 Prijavljeni Bagovi (Bug Reports)

Identifikovani bagovi prilikom testiranja sa korisnikom `problem_user`.

1.3.1 BUG-01: Neispravne slike proizvoda

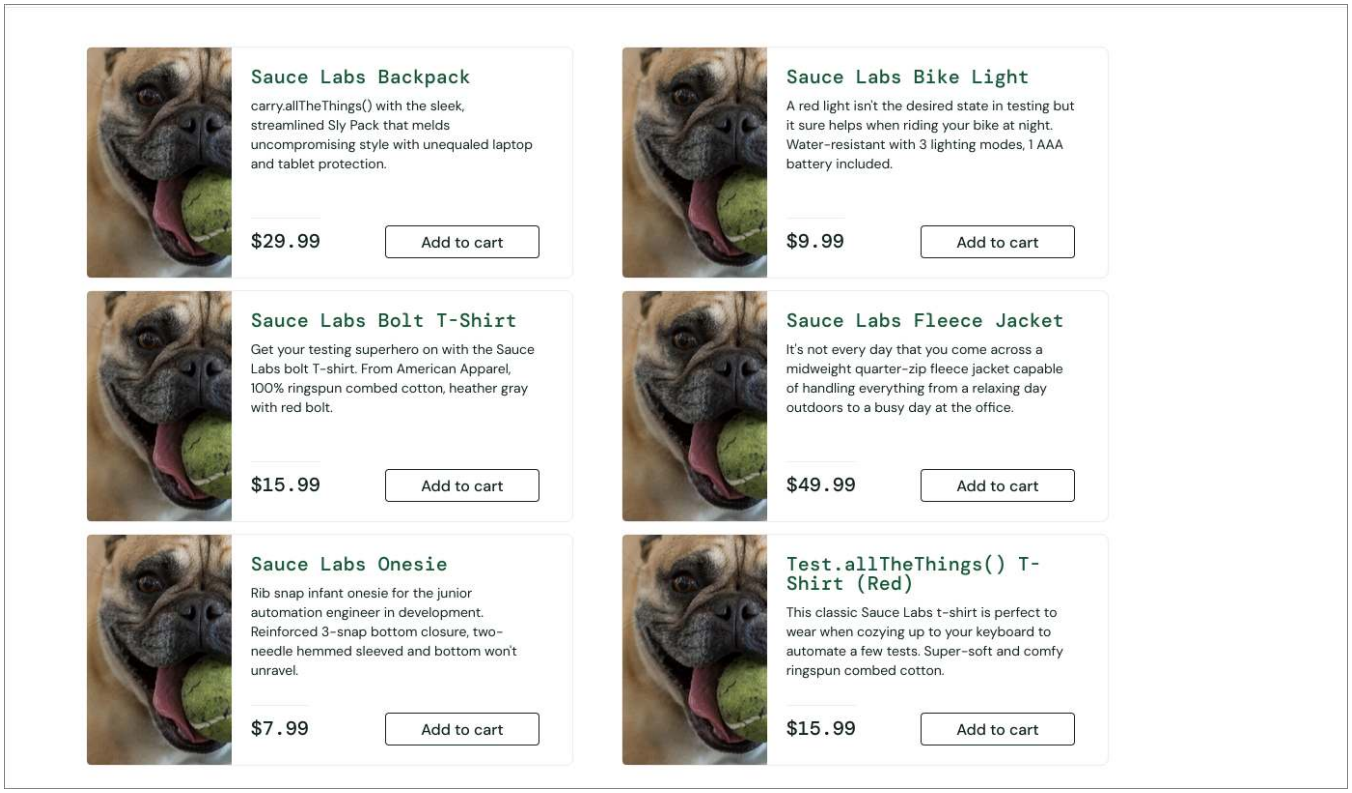
1.3.2 BUG-02: Greška pri unosu prezimena

ID	BUG-01
Naslov	Pogrešne slike proizvoda na Inventory stranici
Environment	Chrome, Windows 11, User: `problem_user`
Severity	2 - High
Koraci za reprodukciju	1. Otvoriti https://www.saucedemo.com/ 2. Prijaviti se sa korisničkim imenom 'problem_user' i lozinkom 'secret_sauce'. 3. Pogledati slike proizvoda na listi (npr. Sauce Labs Backpack).
Očekivani Rezultat	Svaki proizvod treba imati jedinstvenu sliku koja odgovara nazivu.
Aktuelni Rezultat	Svi proizvodi prikazuju istu sliku (slika psa) ili broken image ikonu.
Link	Početna stranica (NAPOMENA: prijaviti se prvo kao problem_user)



Slika 6: Izgled bug-a 1 work item-a na Azure DevOpsu

ID	BUG-02
Naslov	Nemogućnost unosa prezimena na Checkout formi
Environment	Chrome, Windows 11, User: `problem_user`
Severity	1 - Critical
Koraci	1. Prijaviti se kao 'problem_user'. 2. Dodati proizvod u korpu i otići na Checkout. 3. Unijeti validno "First Name" (npr. Test). 4. Unijeti validno "Last Name" (npr. Prezime). 5. Unijeti validan "Zip Code" (npr. 71000). 6. Kliknuti na dugme "Continue".
Očekivani Rezultat	Sistem treba prihvatiti podatke i preći na Overview stranicu.
Aktuelni Rezultat	Sistem prikazuje grešku "Error: Last Name is required" jer je zbog sistemskog buga nemoguće upisati podatke u polje "Last Name"
Link	Stranica za prvi dio checkouta (NAPOMENA: prijaviti se prvo kao problem_user)



Slika 7: Izgled aktuelnog rezultata

Repro Steps

Koraci:
1. Prijaviti se kao 'problem_user'.
2. Dodati proizvod u korpu i otići na Checkout.
3. Unijeti validno "First Name" (npr. Test).
4. Unijeti validno "Last Name" (npr. Prezime).
5. Unijeti validan "Zip Code" (npr. 71000).
6. Kliknuti na dugme "Continue".

Očekivani rezultati:
Sistem treba prihvatiti validnu ekvivalentnu klasu unosa i preusmjeriti korisnika na "Overview" stranicu.

Aktuelni rezultati:
Sistem prikazuje grešku "Error: Last Name is required" jer je zbog sistemskog buga nemoguće upisati podatke u polje "Last Name"

Planning

Resolved Reason

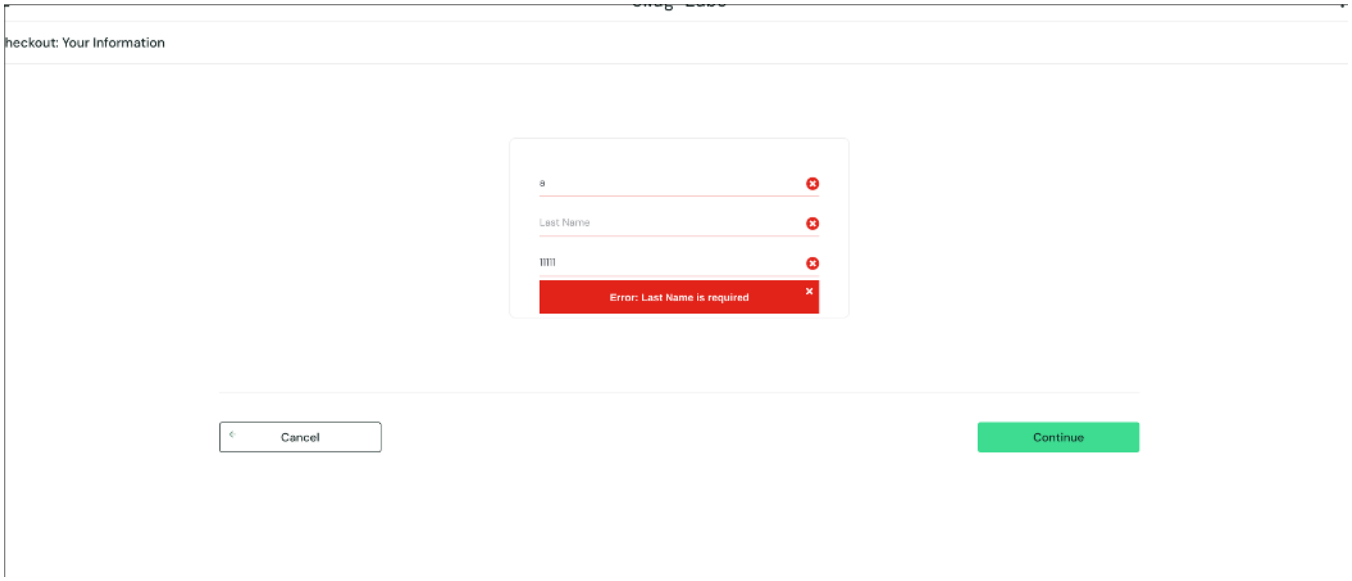
Story Points

Priority
1

Severity
1 - Critical

Activity

Slika 8: Izgled bug-a 2 work item-a na Azure DevOpsu



Slika 9: Izgled aktuelnog rezultata

1.4 Zaključak

Cilj ovog izvještaja bio je provjera funkcionalnog integriteta i stabilnosti web aplikacije **"Swag Labs" (SauceDemo)** primjenom standardizovanih metoda **Black Box testiranja**, u skladu sa teorijskim okvirom definisanim u nastavnim materijalima ("Predavanje 9").

Proces validacije obuhvatio je dizajniranje i izvršavanje pet (5) ključnih testnih slučajeva koji su pokrili vitalne dijelove sistema: proces autentifikacije, upravljanje inventarom i transakcijski ciklus kupovine. Prilikom dizajna testova, primijenjene su tehnike **Ekvivalentnog particionisanja (EP)** za razlikovanje validnih i nevalidnih klasa unosa, te **Analiza graničnih vrijednosti (BVA)** za provjeru rubnih uslova.

Tokom faze izvršavanja testova identifikovana su i dokumentovana dva značajna defekta:

1. **Funkcionalni defekt (Critical):** Onemogućavanje završetka procesa kupovine uslijed neispravne validacije polja "Last Name".
2. **UI/UX defekt (High):** Nekonzistentnost u prikazu vizuelnih elemenata (slike proizvoda).

2 Zadatak 2 – Azure DevOps

U okviru drugog dijela projekta, izvršena je automatizacija testnih slučajeva definisanih u prethodnoj fazi. Proces osiguranja kvaliteta softvera proveden je kroz dvije ključne etape: inicijalna automatizacija korištenjem alata *Selenium IDE*, te razvoj robusnog okvira za automatizaciju (eng. *automation framework*) zasnovanog na bibliotekama *NUnit* i *Selenium* unutar .NET okruženja. Izvorni kod testova i prateća dokumentacija pohranjeni su na repozitorij projekta na platformi *Azure DevOps*.

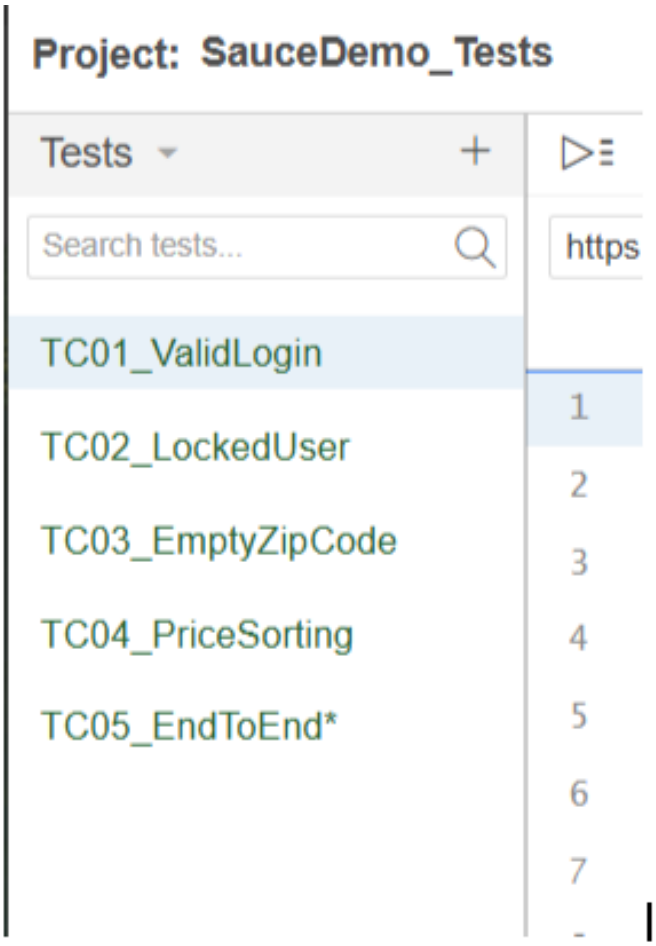
2.1 Automatizacija pomoću Selenium IDE alata

Prva faza podrazumijevala je snimanje i reprodukciju testnih scenarija putem *Selenium IDE* dodatka. Implementirano je pet automatizovanih testnih slučajeva koji pokrivaju ključne funkcionalnosti aplikacije *SauceDemo*:

- **TC01_ValidLogin**: Verifikacija prijave na sistem sa validnim korisničkim podacima.

- **TC02_LockedUser**: Provjera ponašanja sistema prilikom pokušaja prijave zaključanog korisnika.
- **TC03_EmptyZipCode**: Validacija obaveznog polja za poštanski broj tokom procesa kupovine.
- **TC04_PriceSorting**: Testiranje funkcionalnosti sortiranja proizvoda po cijeni (od najniže do najviše).
- **TC05_EndToEnd**: Simulacija kompletnog procesa kupovine, od odabira proizvoda do završetka narudžbe.

Svi implementirani testovi su uspješno izvršeni, čime je potvrđena konzistentnost i ispravnost definisanih testnih scenarija.



Slika 10: Prikaz uspješnog izvršavanja svih testnih slučajeva u Selenium IDE okruženju

2.1.1 Detaljni pregled implementiranih scenarija

Svaki testni slučaj je dizajniran da replicira specifične korisničke akcije. Na primjer, testni slučaj **TC01** započinje inicijalizacijom preglednika (rezolucija 808x912px), nakon čega slijedi unos pristupnih podataka (`standard_user`) i verifikacija uspješnog pristupa sistemu. S druge strane, **TC02** verificira sigurnosni aspekt aplikacije detekcijom poruke o grešci *"Epic sadface: Sorry, this user has been locked out"* prilikom prijave sa korisnikom `locked_out_user`.

https://www.saucedemo.com

	Command	Target	Value
1	open	/	
2	set window size	808x912	
3	click	css="[data-test="username"]"	
4	type	css="[data-test="username"]"	standard_user
5	click	css="[data-test="password"]"	
6	click	css="[data-test="login-password"]"	
7	click	css="[data-test="password"]"	
8	click	css="[data-test="password"]"	
9	type	css="[data-test="password"]"	secret_sauce
10	click	css="[data-test="login-button"]"	
11	close		

Slika 11: TC01 - Prijava sa validnim podacima

https://www.saucedemo.com

	Command	Target	Value
1	open	/	
2	set window size	810x912	
3	click	css="[data-test="username"]"	
4	type	css="[data-test="username"]"	locked_out_user
5	click	css="[data-test="password"]"	
6	click	css="[data-test="login-password"]"	
7	click	css="[data-test="password"]"	
8	click	css="[data-test="password"]"	
9	type	css="[data-test="password"]"	secret_sauce
10	click	css="[data-test="login-button"]"	
11	mouse over	css="[data-test="login-button"]"	
12	mouse out	css="[data-test="login-button"]"	
13	assert text	css="[data-test="error"]"	Epic sadface: Sorry, this user has been locked out.

Slika 12: TC02 - Prijava zaključanog korisnika

Funkcionalna validacija forme za narudžbu demonstrirana je kroz test **TC03**, gdje se namjernim izostavljanjem poštanskog broja izaziva i validira sistemska poruka *"Error: Postal Code is required"*. Testiranje poslovne logike sortiranja (**TC04**) koristi *assert* naredbe za provjeru cijena prvog ($7.99) i posljednjeg ($49.99) artikla na listi.

https://www.saucedemo.com

	Command	Target	Value
1	open	/	
2	set window size	1051x893	
3	click	css="[data-test="username"]"	
4	type	css="[data-test="username"]"	standard_user
5	click	css="[data-test="password"]"	
6	click	css=.login_credentials_wrap-inner	
7	click	css="[data-test="login-password"]"	
8	click	css="[data-test="login-password"]"	
9	click	css="[data-test="login-password"]"	
10	click	css="[data-test="password"]"	
11	type	css="[data-test="password"]"	secret_sauce
12	click	css="[data-test="login-button"]"	
13	click	css="[data-test="add-to-cart-sauce-labs-backpack"]"	
14	click	css="[data-test="shopping-cart-link"]"	
15	click	css="[data-test="checkout"]"	
16	click	css="[data-test="firstName"]"	
17	type	css="[data-test="firstName"]"	Test
18	type	css="[data-test="lastName"]"	User
19	click	css="[data-test="continue"]"	
20	assert text	css="[data-test="error"]"	Error: Postal Code is required
21	click	css="[data-test="shopping-cart-link"]"	
22	click	css="[data-test="remove-sauce-labs-backpack"]"	
23	close		

Slika 13: TC03 - Validacija praznog poštanskog broja

https://www.saucedemo.com

	Command	Target	Value
1	open	/	
2	set window size	814x912	
3	click	css="[data-test="username"]"	
4	type	css="[data-test="username"]"	standard_user
5	click	css="[data-test="password"]"	
6	click	css="[data-test="login-password"]"	
7	click	css="[data-test="password"]"	
8	type	css="[data-test="password"]"	secret_sauce
9	click	css="[data-test="login-button"]"	
10	click	css="[data-test="product-sort-container"]"	
11	select	css="[data-test="product-sort-container"]"	label=Price (low to high)
12	assert text	xpath=(//div[@class='inventory_item_price'])[1]	$7.99
13	assert text	(//div[@class='inventory_item_price'])[6]	$49.99

Slika 14: TC04 - Sortiranje proizvoda po cijeni

Najkompleksniji scenario, **TC05 (End-to-End)**, simulira cjelokupan tok kupovine. Test obuhvata dodavanje proizvoda u korpu, popunjavanje *checkout* forme validnim podacima (Ime: "Test", Prezime: "User", Zip: "72270"), te finalizaciju narudžbe. Uspješnost testa se potvrđuje povratkom na početnu listu proizvoda.

https://www.saucedemo.com

	Command	Target	Value
1	open	/	
2	set window size	1051x893	
3	click	css="[data-test="username"]"	
4	type	css="[data-test="username"]"	standard_user
5	click	css=.login_credentials_wrap-inner	
6	click	css=.login_credentials_wrap-inner	
7	click	css="[data-test="login-password"]"	
8	click	css="[data-test="login-password"]"	
9	double click	css="[data-test="login-password"]"	
10	click	css="[data-test="password"]"	
11	type	css="[data-test="password"]"	secret_sauce
12	click	css="[data-test="login-button"]"	
13	click	css="[data-test="add-to-cart-sauce-labs-backpack"]"	
14	click	css="[data-test="shopping-cart-link"]"	
15	click	css="[data-test="checkout"]"	
16	click	css="[data-test="firstName"]"	
17	type	css="[data-test="firstName"]"	Test
18	type	css="[data-test="lastName"]"	User
19	click	css="[data-test="postalCode"]"	
20	click	css="[data-test="postalCode"]"	
21	type	css="[data-test="postalCode"]"	72270
22	click	css="[data-test="continue"]"	
23	click	css="[data-test="finish"]"	
24	click	css="[data-test="back-to-products"]"	
25	close		

Slika 15: TC05 - End-to-End scenario kupovine u Selenium IDE

2.2 NUnit Selenium Framework

U nastavku projekta napravljen je iskorak ka robusnijem rješenju migracijom testova u *NUnit* okvir i programski jezik C#. Ovakav pristup omogućio je primjenu naprednih dizajnerskih obrazaca, čime je postignuta stvarna modularnost sistema, a sam kod je postao znatno pregledniji i lakši za održavanje.

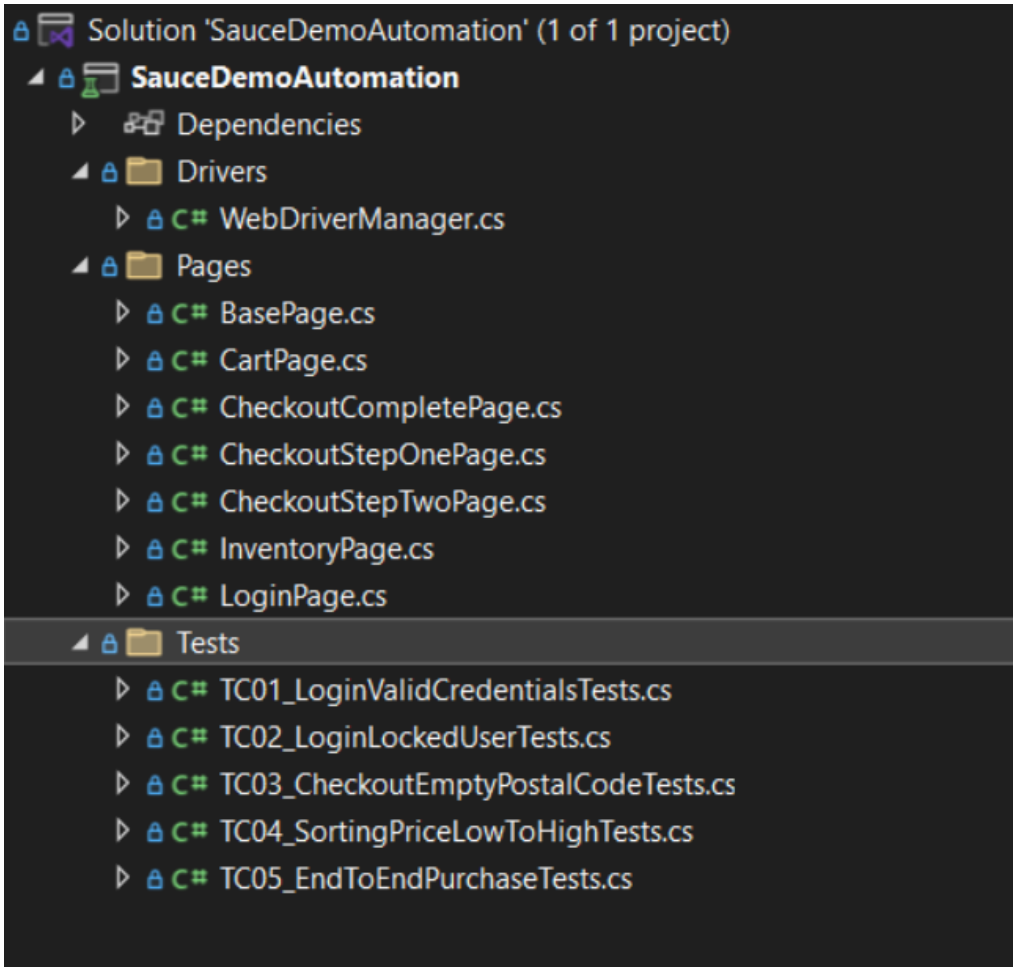
2.2.1 Arhitektura projekta (Design Patterns)

Projektna struktura je organizovana u tri logičke cjeline, primjenjujući **Page Object Model (POM)** i **Singleton** obrasce:

1. **Drivers**: Sadrži klasu `WebDriverManager.cs` koja implementira *Singleton* obrazac. Ovim se osigurava postojanje samo jedne instance *WebDriver*-a tokom izvršavanja testova, čime se optimizuje upotreba sistemskih resursa.

2. **Pages**: Implementira POM obrazac kroz sedam klasa. Klasa `BasePage.cs` sadrži zajedničke metode, dok specifične klase (npr. `LoginPage.cs`, `InventoryPage.cs`, `CartPage.cs`) mapiraju elemente pojedinačnih stranica aplikacije.

3. **Tests**: Sadrži pet klasa koje odgovaraju prethodno definisanim testnim slučajevima. Odvajanjem logike testa od logike interakcije sa stranicom postignut je visok stepen održivosti koda.



Slika 16: Prikaz folder strukture projekta u Visual Studiju sa primjenom POM i Singleton obrazaca

Listing 1: Implementacija Singleton obrasca u WebDriverManager klasi

```
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using WebDriverManager;
using WebDriverManager.DriverConfigs.Impl;

namespace SauceDemoAutomation.Drivers;

/// <summary>
/// Singleton pattern implementation for WebDriver management.
/// Ensures only one browser instance is used across all tests.
/// Uses WebDriverManager.Net for automatic ChromeDriver version matching.
/// </summary>
public sealed class DriverManager
{
    private static readonly Lazy<DriverManager> _instance = new(() => new DriverManager()
        ↪ );
    private IWebDriver? _driver;
    private readonly object _lock = new();

    private DriverManager() { }

    public static DriverManager Instance => _instance.Value;

    public IWebDriver Driver
    {
        get
        {
            lock (_lock)
            {
                if (_driver == null)
                {
                    new WebDriverManager.DriverManager().SetUpDriver(new ChromeConfig());
                    var options = new ChromeOptions();
                    options.AddArgument("--start-maximized");
                    options.AddArgument("--disable-notifications");
                    _driver = new ChromeDriver(options);
                    _driver.Manage().Timeouts().ImplicitWait = TimeSpan.FromSeconds(10);
                }
                return _driver;
            }
        }
    }

    public void QuitDriver()
    {
        lock (_lock)
        {
            if (_driver != null)
            {
                _driver.Quit();
                _driver.Dispose();
                _driver = null;
            }
        }
    }
}
```

Listing 2: LoginPage klasa sa mapiranjem elemenata

```
using OpenQA.Selenium;

namespace SauceDemoAutomation.Pages;

/// <summary>
/// Page Object for the Login page (https://www.saucedemo.com/).
/// </summary>
public class LoginPage : BasePage
{
    // Locators
    private readonly By _usernameField = By.Id("user-name");
    private readonly By _passwordField = By.Id("password");
    private readonly By _loginButton = By.Id("login-button");
    private readonly By _errorMessage = By.CssSelector("[data-test='error']");

    public LoginPage NavigateTo()
    {
        Driver.Navigate().GoToUrl(BaseUrl);
        return this;
    }

    public LoginPage EnterUsername(string username)
    {
        SendKeys(_usernameField, username);
        return this;
    }

    public LoginPage EnterPassword(string password)
    {
        SendKeys(_passwordField, password);
        return this;
    }

    public InventoryPage ClickLoginButton()
    {
        Click(_loginButton);
        return new InventoryPage();
    }

    public LoginPage ClickLoginButtonExpectingError()
    {
        Click(_loginButton);
        WaitForElement(_errorMessage);
        return this;
    }

    public string GetErrorMessage()
    {
        return GetText(_errorMessage);
    }

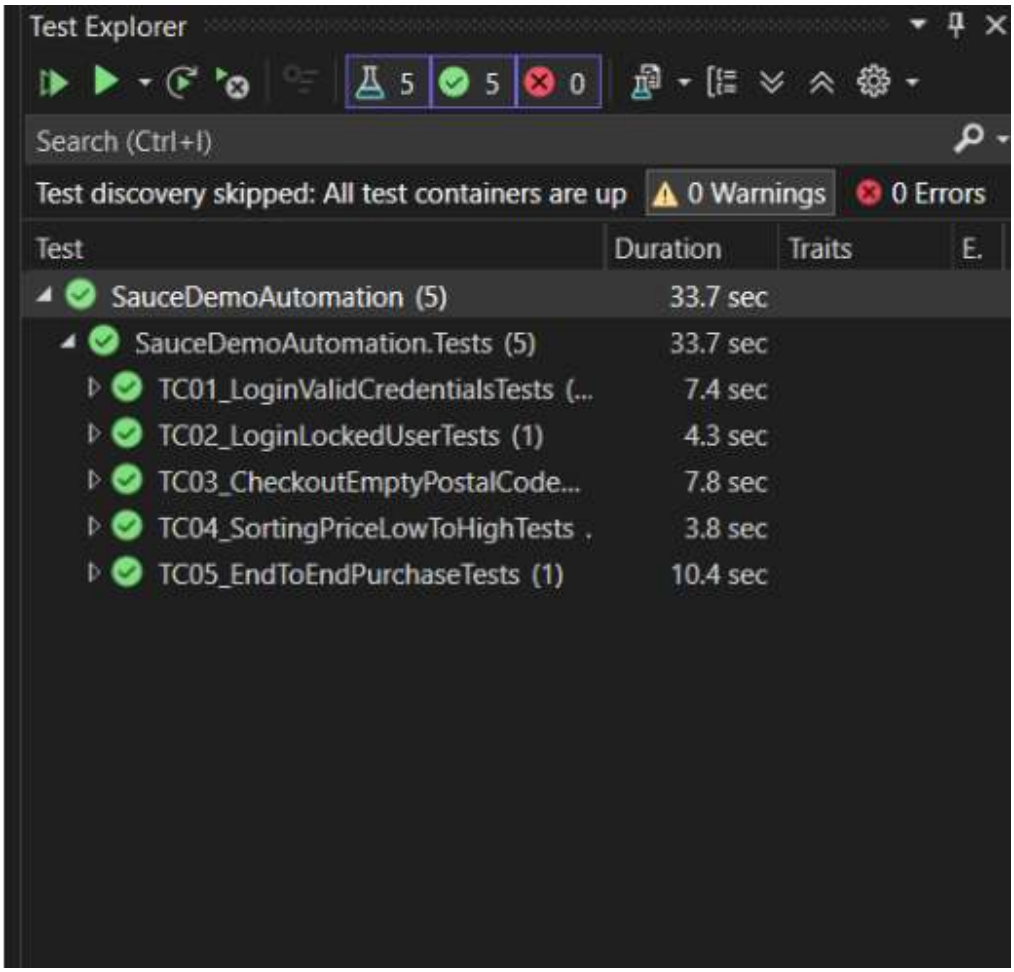
    public bool IsErrorMessageDisplayed()
    {
        return IsElementDisplayed(_errorMessage);
    }

    public InventoryPage LoginAs(string username, string password)
    {
        EnterUsername(username);
        EnterPassword(password);
        return ClickLoginButton();
```

```
    }
}
```

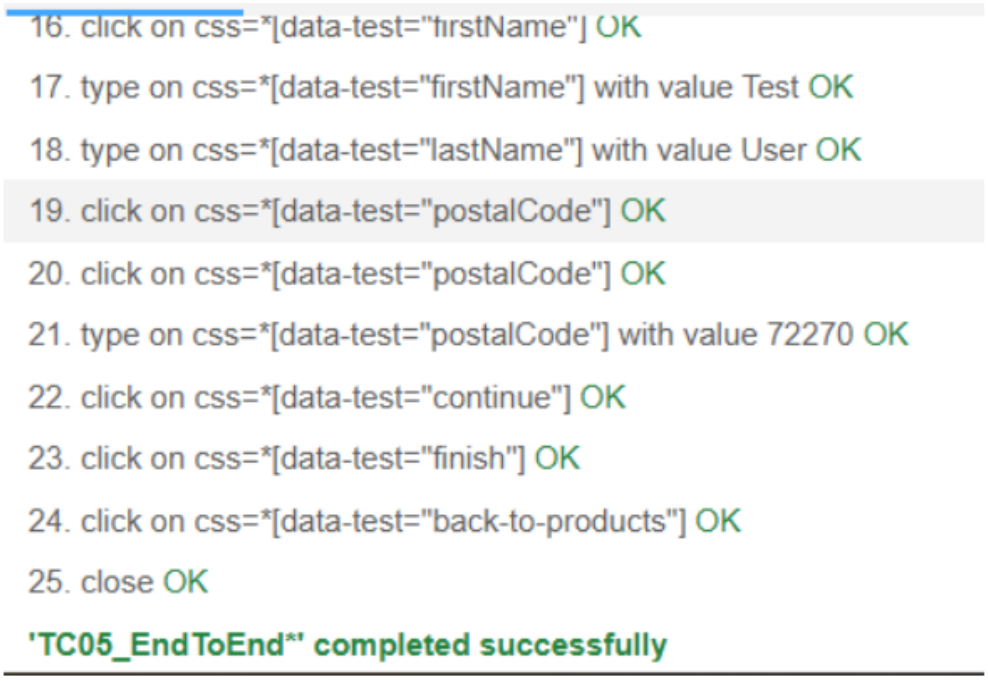
2.2.2 Izvršavanje i rezultati testiranja

Svi implementirani NUnit testovi su uspješno prošli verifikaciju unutar *Test Explorer*-a u Visual Studiju, što potvrđuje da automatizovani scenariji vjerno repliciraju očekivano ponašanje aplikacije.



Slika 17: Prikaz uspješnog prolaska svih NUnit Selenium testova u Test Explorer-u

Prilikom izvršavanja, sistem generiše detaljne logove koji omogućavaju praćenje toka testa. Na primjeru testa **TC05**, log zapisuje sekvencu akcija: unos podataka u polja forme, navigaciju kroz korake *checkout* procesa i finalnu potvrdu uspješnosti operacije.



Slika 18: Primjer generisanog loga nakon uspješne obrade End-to-End testa (TC05)

Listing 3: Primjer NUnit testa za validnu prijavu

```
using SauceDemoAutomation.Drivers;
using SauceDemoAutomation.Pages;

namespace SauceDemoAutomation.Tests;

/// <summary>
/// TC-05: End-to-End kupovina jednog proizvoda
/// Tehnika: Use Case / Scenario testiranje
/// Prioritet: 1 - Critical
/// </summary>
[TestFixture]
public class TC05_EndToEndPurchaseTests
{
    private LoginPage _loginPage = null!;

    [SetUp]
    public void Setup()
    {
        _loginPage = new LoginPage();
        _loginPage.NavigateTo();
    }

    [TearDown]
    public void TearDown()
    {
        DriverManager.Instance.QuitDriver();
    }

    [Test]
    [Category("EndToEnd")]
    [Category("Critical")]
    public void CompletePurchaseOfSingleProduct_ShouldDisplayOrderConfirmation()
```

```
    {
        // Arrange
        const string productName = "Sauce Labs Backpack";
        const string firstName = "Test";
        const string lastName = "User";
        const string postalCode = "71000";
        const string expectedConfirmationMessage = "Thank you for your order!";

        // Act - Step 1: Login
        var inventoryPage = _loginPage.LoginAs("standard_user", "secret_sauce");

        // Act - Step 2: Add product to cart
        inventoryPage.AddItemToCartByName(productName);
        Assert.That(inventoryPage.GetCartItemCount(), Is.EqualTo(1),
            "Cart should have 1 item after adding product");

        // Act - Step 3: Go to cart
        var cartPage = inventoryPage.ClickCartIcon();
        Assert.That(cartPage.IsItemInCart(productName), Is.True,
            $"{productName} should be in the cart");

        // Act - Step 4: Proceed to checkout
        var checkoutStepOne = cartPage.ClickCheckout();

        // Act - Step 5: Fill checkout information
        var checkoutStepTwo = checkoutStepOne.FillCheckoutInfo(firstName, lastName,
            ↪ postalCode);

        // Act - Step 6: Complete the order
        var checkoutComplete = checkoutStepTwo.ClickFinish();

        // Assert
        Assert.Multiple(() =>
        {
            Assert.That(checkoutComplete.GetCurrentUrl(), Does.Contain("checkout-complete
                ↪ .html"),
                "User should be on checkout complete page");
            Assert.That(checkoutComplete.IsOrderComplete(), Is.True,
                "Order confirmation should be displayed");
            Assert.That(checkoutComplete.GetCompleteHeader(), Is.EqualTo(
                ↪ expectedConfirmationMessage),
                "Thank you message should be displayed");
        });
    }
}
```

Link na Azure DevOps projekat: Azure DevOps repozitorij

3 Zadatak 3 – Postman API Testiranje

Za treću stavku projektnog zadatka korištena je Postman kolekcija za testiranje Petstore Swagger API-ja (https://petstore.swagger.io/). Kolekcija sadrži ukupno **63 zahtjeva** organiziranih u tri glavne sekcije: Store (18), Pet (21) i User (24). Svaki zahtjev sadrži detaljne testove implementirane u JavaScriptu koristeći Postman test framework.

3.1 Konfiguracija okruženja

Za izvršavanje testova korištene su sljedeće environment varijable:

- base_url: https://petstore.swagger.io/v2
- base_url_pets: https://petstore.swagger.io/v2/pet
- base_url_store: https://petstore.swagger.io/v2/store
- base_url_user: https://petstore.swagger.io/v2/user
- api_key: special-key (autentifikacijski ključ)

Napomena: Postman alat ima problem sa slanjem slika prilikom izvršavanja kolekcije u GUI-u – testovi padaju jer se ne prosljeđuje raw binary fajl kako stranica očekuje. Međutim, u CLI-u testovi uredno prolaze.

3.2 Pokretanje kolekcije testova

Za pokretanje kolekcije testova koristi se Newman CLI alat, njega je isto tako potrebno instalirati prije pokretanja naredbe za testiranje:

```
npm install -g newman
newman run petstore-collection.json -e petstore-environment.json
```

3.3 Store – Upravljanje narudžbama

Store sekcija sadrži **18 zahtjeva** za testiranje endpointa vezanih za inventar i narudžbe u pet store-u.

3.3.1 Get Inventory

Cilj testova: Verifikovati da endpoint `GET /store/inventory` ispravno vraća mapu statusa inventara. Testovi provjeravaju da je odgovor validan JSON objekt, da su sve vrijednosti nenegativni cijeli brojevi, te da poznati statusi (available, pending, sold) imaju ispravne vrijednosti.

HTTP zahtjev:

```
GET {{base_url_store}}/inventory
Header: api_key: {{api_key}}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (<2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Content-Type is application/json", function () {
    pm.response.to.have.header("Content-Type");
    pm.expect(pm.response.headers.get("Content-Type"))
      .to.include("application/json");
});
```

```
pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Response body is an object (inventory map)", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an("object");
});

pm.test("All inventory values are non-negative integers", function () {
    const jsonData = pm.response.json();
    for (let status in jsonData) {
        pm.expect(jsonData[status]).to.be.a("number");
        pm.expect(jsonData[status]).to.be.at.least(0);
        pm.expect(Number.isInteger(jsonData[status])).to.be.true;
    }
});

pm.test("Known statuses have valid counts (if present)", function () {
    const jsonData = pm.response.json();
    const knownStatuses = ["available", "pending", "sold"];
    knownStatuses.forEach(status => {
        if (jsonData.hasOwnProperty(status)) {
            pm.expect(jsonData[status]).to.be.a("number");
            console.log(status + ": " + jsonData[status]);
        }
    });
});

pm.test("Inventory is not empty", function () {
    const jsonData = pm.response.json();
    pm.expect(Object.keys(jsonData).length).to.be.above(0);
});
```

Objašnjenje testova:

- **Test 1-2:** Osnovni funkcionalni testovi - status 200 i vrijeme odgovora ispod 2 sekunde.
- **Test 3-4:** Validacija formata - Content-Type header i parsibilnost JSON-a.
- **Test 5:** Strukturalna validacija - odgovor mora biti objekt.
- **Test 6:** Iteracija kroz sve ključeve inventara i provjera da su vrijednosti nenegativni cijeli brojevi.
- **Test 7:** Provjera poznatih statusa (available, pending, sold) ako postoje.
- **Test 8:** Inventar ne smije biti prazan - osigurava da API vraća stvarne podatke.

3.3.2 POST /store/order – Kreiranje narudžbe

Place Order - Valid (Happy Path) **Cilj testova:** Kreirati novu narudžbu za ljubimca i validirati strukturu odgovora.

HTTP zahtjev:

```
POST {{base_url_store}}/order
Content-Type: application/json
```

Pre-request skripta:

```
const petId = Math.floor(Math.random() * 900000) + 100000;
pm.environment.set("store_test_pet_id", petId);
console.log("Creating pet with ID: " + petId + " for order test");
```

Body:

```
{
  "id": 0,
  "petId": {{store_test_pet_id}},
  "quantity": 1,
  "shipDate": "2026-01-03T20:00:00.000Z",
  "status": "placed",
  "complete": true
}
```

Post-response testovi:

```
pm.test("Response received for order placement", function () {
    pm.expect([200, 400]).to.include(pm.response.code);
    console.log("Place order response: " + pm.response.code);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Content-Type is application/json", function () {
    pm.expect(pm.response.headers.get("Content-Type"))
      .to.include("application/json");
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

if (pm.response.code === 200) {
    const jsonData = pm.response.json();

    pm.test("Response contains all required fields", function () {
        pm.expect(jsonData).to.have.property("id");
        pm.expect(jsonData).to.have.property("petId");
        pm.expect(jsonData).to.have.property("quantity");
        pm.expect(jsonData).to.have.property("status");
        pm.expect(jsonData).to.have.property("complete");
    });

    pm.test("Order ID is generated and saved", function () {
        pm.expect(jsonData.id).to.be.a("number");
        pm.expect(jsonData.id).to.be.above(0);
        pm.environment.set("order_id", jsonData.id);
        console.log("Order ID saved: " + jsonData.id);
    });
```

```
    pm.test("Quantity matches requested value", function () {
        pm.expect(jsonData.quantity).to.eql(1);
    });

    pm.test("Status is one of valid values", function () {
        const validStatuses = ["placed", "approved", "delivered"];
        pm.expect(validStatuses).to.include(jsonData.status);
    });

    pm.test("Complete field is boolean", function () {
        pm.expect(jsonData.complete).to.be.a("boolean");
    });
}

if (pm.response.code === 400) {
    pm.test("Order rejected (pet may not exist)", function () {
        console.log("Order rejected - expected if pet doesn't exist");
        pm.environment.set("order_id", "1");
    });
}
```

Objašnjenje testova:

- **Test 1:** Prihvaća 200 (uspjeh) ili 400 (pet ne postoji) – dokumentira ponašanje API-ja.
- **Test 2:** Provjera performansi – odgovor mora stići unutar 2 sekunde.
- **Test 3-4:** Validacija formata odgovora.
- **Test 5:** Strukturalna validacija – provjerava sva obavezna polja narudžbe.
- **Test 6:** Sprema order_id u environment za chain testing.
- **Test 7-8:** Validacija vrijednosti polja.

Place Order - Invalid Quantity (Negativna vrijednost) **Cilj testova:** Boundary value testing – testira rubne slučajeve i nevalidne inpute.

Body:

```
{
    "id": 0,
    "petId": 1,
    "quantity": -5,
    "shipDate": "2026-01-03T20:00:00.000Z",
    "status": "placed",
    "complete": false
}
```

Post-response testovi:

```
pm.test("Response received (documenting API behavior)", function () {
    pm.expect([200, 400]).to.include(pm.response.code);
    console.log("API returned: " + pm.response.code + " for negative quantity");
});
```

```
pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("WARNING: API accepted negative quantity (possible bug)", function () {
        const jsonData = pm.response.json();
        console.log("API accepted order with negative quantity: "
                    + jsonData.quantity);
        pm.expect(jsonData.quantity).to.eql(-5);
    });
}

if (pm.response.code === 400) {
    pm.test("API correctly rejected invalid order", function () {
        pm.response.to.have.status(400);
    });
}
```

Objašnjenje testova:

- Dokumentira ponašanje API-ja sa nevalidnim podacima.
- Ako API prihvati negativnu količinu, označava se kao potencijalni bug.

Place Order - Invalid Status **Cilj testova:** Dokumentirati da li API validira enum vrijednosti za status polje.
Body: Status postavljen na "INVALID_STATUS_XYZ"
Post-response testovi:

```
pm.test("Response received for invalid status", function () {
    pm.expect([200, 400, 500]).to.include(pm.response.code);
    console.log("API returned: " + pm.response.code + " for invalid status");
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API behavior with invalid status", function () {
        const jsonData = pm.response.json();
        console.log("Status returned: " + jsonData.status);
    });
}
```

Place Order - Empty Body **Cilj testova:** Provjeriti ponašanje API-ja sa praznim JSON objektom.
Body: `{}`
Post-response testovi:

```
pm.test("Response received for empty body", function () {
    pm.expect([200, 400, 500]).to.include(pm.response.code);
    console.log("API response for empty body: " + pm.response.code);
```

```
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API creates order with defaults", function () {
        const jsonData = pm.response.json();
        console.log("Created order: " + JSON.stringify(jsonData));
        pm.expect(jsonData).to.have.property("id");
    });
}

if (pm.response.code === 400) {
    pm.test("API correctly rejects empty request", function () {
        pm.response.to.have.status(400);
    });
}
```

3.3.3 GET /store/order/{orderId} – Dohvatanje narudžbe

Get Order - Valid ID (Happy Path) **Cilj testova:** Dohvatiti narudžbu po ID-u i validirati sva polja.
HTTP zahtjev:

```
GET {{base_url_store}}/order/{{order_id}}
```

Post-response testovi:

```
const orderId = pm.environment.get("order_id");
console.log("Fetching order with ID: " + orderId);

pm.test("Response received for order lookup", function () {
    pm.expect([200, 404, 405]).to.include(pm.response.code);
    console.log("Get order response: " + pm.response.code);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Content-Type is application/json", function () {
        pm.expect(pm.response.headers.get("Content-Type"))
          .to.include("application/json");
    });

    pm.test("Response is valid JSON", function () {
        pm.response.to.be.json;
    });

    pm.test("Response has correct schema", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData).to.have.property("id");
        pm.expect(jsonData).to.have.property("petId");
```

```
        pm.expect(jsonData).to.have.property("quantity");
        pm.expect(jsonData).to.have.property("status");
    });

    pm.test("Order ID matches requested ID", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData.id).to.eql(parseInt(orderId));
    });
}

if (pm.response.code === 404) {
    pm.test("Order not found (may have been auto-cleaned)", function () {
        console.log("Order not found - demo API may auto-clean orders");
    });
}
```

Objašnjenje testova:

- **Test 1:** Prihvaća 200 (pronađena), 404 (nije pronađena) ili 405 (mogući API quirk).
- **Test 2:** Provjera performansi – odgovor unutar 2 sekunde.
- **Test 3-4:** Validacija formata i JSON-a.
- **Test 5:** Strukturalna validacija – sva obavezna polja narudžbe.
- **Test 6:** Verifikacija da vraćeni ID odgovara traženom ID-u.

Get Order - Boundary ID 1 (Minimalna validna vrijednost) **Cilj testova:** Boundary value testing – testira minimalnu validnu vrijednost (ID=1).

Pre-request skripta:

```
const orderData = {
    "id": 1,
    "petId": 1,
    "quantity": 1,
    "shipDate": new Date().toISOString(),
    "status": "placed",
    "complete": true
};

pm.sendRequest({
    url: pm.environment.get("base_url_store") + "/order",
    method: "POST",
    header: { "Content-Type": "application/json" },
    body: { mode: "raw", raw: JSON.stringify(orderData) }
}, function (err, res) {
    if (err) {
        console.log("Pre-request: Could not create order 1");
    } else {
        console.log("Pre-request: Order 1 created/exists");
    }
});
```

Post-response testovi:

```
pm.test("Status code is 200 for ID = 1 (boundary)", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response is valid JSON with order data", function () {
    pm.response.to.be.json;
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
    pm.expect(jsonData.id).to.eql(1);
});

console.log("Boundary test: ID = 1 (minimum valid) passed");
```

Objašnjenje testova: Testira minimalnu vrijednost ID-a. Pre-request skripta osigurava da narudžba sa ID=1 postoji prije izvršavanja GET zahtjeva. Međutim, server ponekad nema ovaj ID iako je na stranici označen kao validan, što može dovesti do 404 odgovora. Zbog toga testovi prihvaćaju [200, 404] kako bi bili robusni na varijabilno ponašanje shared platforme.

Get Order - Boundary ID 10 (Maksimalna validna vrijednost) **Cilj testova:** Boundary testing – prema dokumentaciji, ID > 10 trebaju generisati greske.

Post-response testovi:

```
pm.test("Response received for ID = 10", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Response for ID=10: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Order data returned for ID 10", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData).to.have.property("id");
        pm.expect(jsonData.id).to.eql(10);
    });
}

console.log("Boundary test: ID = 10 completed");
```

Get Order - Invalid ID 0 **Cilj testova:** Negative testing – ID=0 je invalidan.

Post-response testovi:

```
pm.test("Status code indicates error for ID = 0", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
```

```
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Error response received", function () {
    console.log("Response for ID=0: " + pm.response.code);
});
```

Get Order - Invalid ID 11 (ID > 10) **Cilj testova:** Prema API dokumentaciji, ID > 10 trebaju vratiti gresku.
Post-response testovi:

```
pm.test("Status code indicates error for ID > 10", function () {
    pm.expect([400, 404, 500]).to.include(pm.response.code);
    console.log("Response for ID=11: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 404) {
    pm.test("Order not found message received", function () {
        pm.response.to.have.status(404);
    });
}
```

Get Order - Negative ID **Cilj testova:** Negative testing – negativne ID vrijednosti trebaju biti odbijene.
Post-response testovi:

```
pm.test("Status code indicates error for negative ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("Response for ID=-1: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects negative order ID", function () {
    pm.expect(pm.response.code).to.not.eql(200);
});
```

Get Order - String ID **Cilj testova:** Negative testing – string vrijednosti nisu validne za ID.
Post-response testovi:

```
pm.test("Status code indicates error for string ID", function () {
    pm.expect([400, 404, 500]).to.include(pm.response.code);
    console.log("Response for ID='abc': " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects non-integer order ID", function () {
    pm.expect(pm.response.code).to.not.eql(200);
});

if (pm.response.code !== 200) {
    pm.test("Error response is JSON or documented format", function () {
        try {
            pm.response.to.be.json;
            console.log("Error response: " + pm.response.text());
        } catch (e) {
            console.log("Non-JSON error response: " + pm.response.text());
        }
    });
}
```

Get Order - Very Large ID **Cilj testova:** Provjeriti da API pravilno rukuje integer overflow situacijama.

Post-response testovi:

```
pm.test("Status code indicates error for very large ID", function () {
    pm.expect([400, 404, 500]).to.include(pm.response.code);
    console.log("Response for very large ID: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API handles integer overflow gracefully", function () {
    pm.expect(pm.response.code).to.not.eql(200);
});
```

3.3.4 DELETE /store/order/{orderId} – Brisanje narudžbe

Delete Order - Valid (Happy Path) **Cilj testova:** Obrisati narudžbu i validirati odgovor.

HTTP zahtjev:

```
DELETE {{base_url_store}}/order/{{order_id}}
```

Post-response testovi:

```
const orderId = pm.environment.get("order_id");
```

```
pm.environment.set("deleted_order_id", orderId);
console.log("Attempting to delete order: " + orderId);

pm.test("Delete request processed", function () {
    pm.expect([200, 404, 405]).to.include(pm.response.code);
    console.log("Delete order response: " + pm.response.code);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Order deletion successful", function () {
        console.log("Order " + orderId + " deleted");
    });
}

if (pm.response.code === 404) {
    pm.test("Order already gone (demo API quirk)", function () {
        console.log("Order not found for deletion");
    });
}
```

Get Order - Verify Deleted **Cilj testova:** Verifikacija da je narudžba zaista obrisana.

HTTP zahtjev:

```
GET {{base_url_store}}/order/{{deleted_order_id}}
```

Post-response testovi:

```
const deletedOrderId = pm.environment.get("deleted_order_id");
console.log("Verifying order " + deletedOrderId + " was deleted");

pm.test("Verification request processed", function () {
    pm.expect([404, 405]).to.include(pm.response.code);
    console.log("Verify deleted response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 404) {
    pm.test("Order correctly returns 404 after deletion", function () {
        console.log("Order confirmed deleted");
    });
}
```

Delete Order - Already Deleted (Idempotency Test) **Cilj testova:** Verifikacija idempotentnosti DELETE operacije.

Post-response testovi:

```
const deletedOrderId = pm.environment.get("deleted_order_id");
console.log("Attempting second delete of order: " + deletedOrderId);

pm.test("Second delete request processed", function () {
    pm.expect([404, 405]).to.include(pm.response.code);
    console.log("Second delete response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Idempotency test completed", function () {
    console.log("Delete idempotency check completed");
});
```

Delete Order - Negative ID **Cilj testova:** Negative testing – API trebao bi odbiti negativne ID-jeve.
Post-response testovi:

```
pm.test("Status code indicates error for negative ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("Response for delete ID=-1: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects negative order ID for deletion", function () {
    pm.expect(pm.response.code).to.not.eql(200);
});
```

Delete Order - String ID **Cilj testova:** Negative testing – string vrijednosti nisu validne za ID.
Post-response testovi:

```
pm.test("Status code indicates error for string ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("Response for delete ID='invalid': " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects non-integer ID for deletion", function () {
    pm.expect(pm.response.code).to.not.eql(200);
});
```

3.4 Pet – Upravljanje ljubimcima

Pet sekcija sadrži **21 zahtjev** za testiranje svih CRUD operacija nad ljubimcima.

3.4.1 POST /pet – Kreiranje ljubimca

Create Pet - Valid (Happy Path) **Cilj testova:** Kreirati novog ljubimca sa svim poljima i validirati da API ispravno sprema sve podatke.

Pre-request skripta:

```
const timestamp = Date.now();
const uniquePetName = "TestPet_" + timestamp;
const petId = Math.floor(Math.random() * 900000) + 100000;

pm.environment.set("pet_name", uniquePetName);
pm.environment.set("new_pet_id", petId);

console.log("Pet name set to: " + uniquePetName);
console.log("Pet ID set to: " + petId);
```

HTTP zahtjev:

```
POST {{base_url_pets}}
Content-Type: application/json
```

Body:

```
{
  "id": {{new_pet_id}},
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "{{pet_name}}",
  "photoUrls": [
    "https://example.com/photo1.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "friendly"
    }
  ],
  "status": "available"
}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
```

```
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Response contains all pet fields", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
    pm.expect(jsonData).to.have.property("category");
    pm.expect(jsonData).to.have.property("name");
    pm.expect(jsonData).to.have.property("photoUrls");
    pm.expect(jsonData).to.have.property("tags");
    pm.expect(jsonData).to.have.property("status");
});

pm.test("Pet ID is saved correctly", function () {
    const jsonData = pm.response.json();
    const expectedId = pm.environment.get("new_pet_id");

    pm.environment.set("pet_id", jsonData.id);
    console.log("Pet ID saved: " + jsonData.id);
    pm.expect(jsonData.id).to.be.a("number");
});

pm.test("Pet name matches request", function () {
    const jsonData = pm.response.json();
    const expectedName = pm.environment.get("pet_name");
    pm.expect(jsonData.name).to.eql(expectedName);
});

pm.test("Status is valid enum value", function () {
    const jsonData = pm.response.json();
    const validStatuses = ["available", "pending", "sold"];
    pm.expect(validStatuses).to.include(jsonData.status);
});

pm.test("PhotoUrls is an array", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.photoUrls).to.be.an("array");
});

pm.test("Tags is an array", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.tags).to.be.an("array");
});

pm.test("Category has correct structure", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.category).to.have.property("id");
    pm.expect(jsonData.category).to.have.property("name");
});
```

Objašnjenje testova:

- **Test 1-3:** Osnovna validacija statusa, vremena i JSON formata.

- **Test 4:** Provjera da odgovor sadrži sva obavezna polja pet objekta.
- **Test 5:** Sprema pet_id u environment za chain testing i verifikaciju.
- **Test 6:** Potvrda da je ime ljubimca ispravno pohranjeno.
- **Test 7:** Validacija enumeracije status vrijednosti.
- **Test 8-9:** Provjera da su photoUrls i tags polja zaista nizovi.
- **Test 10:** Strukturalna validacija category objekta.

Create Pet - Minimal (Samo obavezna polja) **Cilj testova:** Verifikacija da API prihvata minimalan skup polja.

Pre-request skripta:

```
const minimalPetId = Math.floor(Math.random() * 900000) + 200000;
pm.environment.set("minimal_pet_id_request", minimalPetId);
console.log("Minimal pet ID: " + minimalPetId);
```

Body:

```
{
  "id": {{minimal_pet_id_request}},
  "name": "MinimalPet",
  "photoUrls": [
    "https://example.com/minimal.jpg"
  ]
}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Pet created with minimal fields", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
    pm.expect(jsonData).to.have.property("name");
    pm.expect(jsonData.name).to.eql("MinimalPet");

    pm.environment.set("minimal_pet_id", jsonData.id);
    console.log("Minimal pet created with ID: " + jsonData.id);
});
```

```
pm.test("PhotoUrls preserved", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.photoUrls).to.be.an("array");
    pm.expect(jsonData.photoUrls.length).to.be.above(0);
});
```

Objašnjenje testova: Testira da API ispravno rukuje sa minimalnim setom podataka – samo id, name i photoUrls.

Create Pet - Empty Body **Cilj testova:** Dokumentirati ponašanje sa praznim JSON objektom.

Body: `{}`

Post-response testovi:

```
pm.test("Response received for empty body", function () {
    pm.expect([200, 400, 405, 500]).to.include(pm.response.code);
    console.log("Empty body response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("WARNING: API accepts empty pet body", function () {
        console.log("API created pet with empty body - potential issue");
        const jsonData = pm.response.json();
        console.log("Created pet: " + JSON.stringify(jsonData));
    });
}

if (pm.response.code === 400 || pm.response.code === 405) {
    pm.test("API correctly rejects empty body", function () {
        console.log("API requires pet data");
    });
}
```

Create Pet - Missing Required Fields **Cilj testova:** Testiranje validacije obaveznih polja (name, photoUrls).

Body:

```
{
  "id": 12345,
  "status": "available"
}
```

Post-response testovi:

```
pm.test("Response received for missing required fields", function () {
    pm.expect([200, 400, 405]).to.include(pm.response.code);
    console.log("Missing required fields response: " + pm.response.code);
```

```
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API accepts pet without name/photoUrls", function () {
        console.log("API does not enforce required fields");
    });
}

if (pm.response.code === 400 || pm.response.code === 405) {
    pm.test("API correctly validates required fields", function () {
        console.log("API requires name and photoUrls");
    });
}
```

3.4.2 GET /pet/{petId} – Dohvatanje ljubimca

Get Pet - Valid ID (Happy Path) **Cilj testova:** Dohvatiti ljubimca po ID-u i validirati sva polja.
HTTP zahtjev:

```
GET {{base_url_pets}}/{{pet_id}}
```

Post-response testovi:

```
const petId = pm.environment.get("pet_id");
console.log("Fetching pet with ID: " + petId);

pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Content-Type is application/json", function () {
    pm.expect(pm.response.headers.get("Content-Type"))
      .to.include("application/json");
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Response contains all pet fields", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
    pm.expect(jsonData).to.have.property("name");
    pm.expect(jsonData).to.have.property("photoUrls");
    pm.expect(jsonData).to.have.property("status");
});
```

```
pm.test("Pet ID matches requested ID", function () {
    const jsonData = pm.response.json();
    const requestedId = parseInt(pm.environment.get("pet_id"));
    pm.expect(jsonData.id).to.eql(requestedId);
});

pm.test("Data types are correct", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.id).to.be.a("number");
    pm.expect(jsonData.name).to.be.a("string");
    pm.expect(jsonData.photoUrls).to.be.an("array");
    pm.expect(jsonData.status).to.be.a("string");
});

pm.test("Status is valid enum", function () {
    const jsonData = pm.response.json();
    const validStatuses = ["available", "pending", "sold"];
    pm.expect(validStatuses).to.include(jsonData.status);
});
```

Objašnjenje testova:

- **Test 1-3:** Osnovna validacija statusa, vremena i formata.
- **Test 4-5:** Validacija JSON formata i strukture odgovora.
- **Test 6:** Potvrda da vraćeni ID odgovara traženom.
- **Test 7:** Provjera tipova podataka svih polja.
- **Test 8:** Validacija enumeracije status vrijednosti.

Get Pet - Invalid ID String **Cilj testova:** Negative testing – string vrijednosti nisu validne za ID.
HTTP zahtjev:

```
GET {{base_url_pets}}/notanumber
```

Post-response testovi:

```
pm.test("Status code indicates error for string ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("String ID response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects non-integer pet ID", function () {
    pm.expect(pm.response.code).to.not.eql(200);
    console.log("API correctly rejects string ID");
});
```

Get Pet - Not Found **Cilj testova:** Validacija 404 odgovora za nepostojećeg ljubimca.
HTTP zahtjev:

```
GET {{base_url_pets}}/999999999999
```

Post-response testovi:

```
pm.test("Status code is 404 for non-existent pet", function () {
    pm.response.to.have.status(404);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Error response structure", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData).to.have.property("type");
    pm.expect(jsonData).to.have.property("message");
});

pm.test("Error message indicates pet not found", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.message).to.include("Pet not found");
});
```

Get Pet - Negative ID **Cilj testova:** Negative testing – negativne ID vrijednosti trebaju biti odbijene.
HTTP zahtjev:

```
GET {{base_url_pets}}/-1
```

Post-response testovi:

```
pm.test("Status code indicates error for negative ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("Negative ID response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API handles negative ID appropriately", function () {
    pm.expect(pm.response.code).to.not.eql(200);
    console.log("Negative ID handled correctly");
});
```

3.4.3 GET /pet/findByStatus – Pretraga ljubimaca po statusu

Find Pets By Status - Available **Cilj testova:** Testirati filtriranje ljubimaca po statusu.
HTTP zahtjev:

```
GET {{base_url_pets}}/findByStatus?status=available
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 3000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(3000);
});

pm.test("Response is valid JSON array", function () {
    pm.response.to.be.json;
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an("array");
});

pm.test("All returned pets have 'available' status", function () {
    const jsonData = pm.response.json();

    if (jsonData.length > 0) {
        jsonData.forEach((pet, index) => {
            pm.expect(pet.status).to.eql("available");
        });
        console.log("Checked " + jsonData.length + " pets - all available");
    } else {
        console.log("No available pets found");
    }
});

pm.test("Returned pets have required structure", function () {
    const jsonData = pm.response.json();

    if (jsonData.length > 0) {
        const firstPet = jsonData[0];
        pm.expect(firstPet).to.have.property("id");
        pm.expect(firstPet).to.have.property("name");
        pm.expect(firstPet).to.have.property("photoUrls");
    }
});
```

Objašnjenje testova:

- **Test 1-3:** Osnovna validacija statusa, vremena i JSON niza.
- **Test 4:** Iteracija kroz sve vraćene ljubimce i provjera da svi imaju status "available".
- **Test 5:** Provjera da su svi vraćeni objekti ispravne strukture.

Find Pets By Status - Multiple Statuses **Cilj testova:** Testiranje filtriranja sa više statusa.

HTTP zahtjev:

```
GET {{base_url_pets}}/findByStatus?status=available&status=pending
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(3000);
});

pm.test("Response is valid JSON array", function () {
    pm.response.to.be.json;
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an("array");
});

pm.test("Returned pets have valid statuses", function () {
    const jsonData = pm.response.json();
    const validStatuses = ["available", "pending"];

    if (jsonData.length > 0) {
        jsonData.forEach((pet) => {
            pm.expect(validStatuses).to.include(pet.status);
        });
        console.log("All " + jsonData.length + " pets have valid status");
    }
});
```

Find Pets By Status - Invalid Status **Cilj testova:** Dokumentirati ponašanje za nevalidne status vrijednosti.

HTTP zahtjev:

```
GET {{base_url_pets}}/findByStatus?status=invalidstatus
```

Post-response testovi:

```
pm.test("Response received for invalid status", function () {
    pm.expect([200, 400]).to.include(pm.response.code);
    console.log("Invalid status response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API returns empty array for invalid status", function () {
```

```
        const jsonData = pm.response.json();
        console.log("Returned " + jsonData.length + " pets for invalid status");
    });
}

if (pm.response.code === 400) {
    pm.test("API correctly rejects invalid status", function () {
        console.log("API validates status values");
    });
}
```

3.4.4 GET /pet/findByTags – Pretraga po tagovima (DEPRECATED)

Napomena: Ovaj endpoint je označen kao deprecated u OpenAPI specifikaciji.
HTTP zahtjev:

```
GET {{base_url_pets}}/findByTags?tags=friendly
```

Post-response testovi:

```
pm.test("Deprecated endpoint responds", function () {
    pm.expect([200, 400, 404, 410]).to.include(pm.response.code);
    console.log("Deprecated findByTags response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(3000);
});

if (pm.response.code === 200) {
    pm.test("Deprecated endpoint still functional", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData).to.be.an("array");
        console.log("DEPRECATED: findByTags still works, returns "
                    + jsonData.length + " pets");
    });
}

if (pm.response.code === 410) {
    pm.test("Endpoint correctly returns 410 Gone", function () {
        console.log("Deprecated endpoint properly returns 410 Gone");
    });
}

pm.test("Document deprecation status", function () {
    console.log("NOTE: This endpoint is marked as deprecated");
});
```

3.4.5 PUT /pet – Azuriranje ljubimca

Cilj testova: Ažurirati postojećeg ljubimca sa novim podacima.
HTTP zahtjev:

```
PUT {{base_url_pets}}
Content-Type: application/json
```

Body:

```
{
  "id": {{pet_id}},
  "category": {"id": 1, "name": "Dogs"},
  "name": "UpdatedPetName",
  "photoUrls": ["https://example.com/updated-photo.jpg"],
  "tags": [{"id": 1, "name": "updated"}],
  "status": "sold"
}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Pet name was updated", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.name).to.eql("UpdatedPetName");
    console.log("Pet name updated to: " + jsonData.name);
});

pm.test("Pet status was updated to sold", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.status).to.eql("sold");
    console.log("Pet status updated to: " + jsonData.status);
});

pm.test("Pet ID unchanged", function () {
    const jsonData = pm.response.json();
    const originalId = parseInt(pm.environment.get("pet_id"));
    pm.expect(jsonData.id).to.eql(originalId);
});
```

3.4.6 POST /pet/{petId} – Ažuriranje putem forme

Cilj testova: Ažuriranje ljubimca koristeći form data umjesto JSON-a.
HTTP zahtjev:

```
POST {{base_url_pets}}/{{pet_id}}
```

```
Content-Type: application/x-www-form-urlencoded
```

Body:

```
name=FormUpdatedPet&status=pending
```

Post-response testovi:

```
pm.test("Response received for form data update", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Form data update response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Form data update successful", function () {
        console.log("Pet updated via form data");
    });
}

if (pm.response.code === 404) {
    pm.test("Pet not found for form update (demo API quirk)", function () {
        console.log("Pet not found - this is acceptable for demo API");
    });
}
```

Objašnjenje testova: Testira alternativni format ažuriranja – URL-encoded forma umjesto JSON.

Get Pet - Verify Update **Cilj testova:** Verifikacija da su podaci zaista azurirani.
Post-response testovi:

```
pm.test("Response received for update verification", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Verify update response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Pet data retrieved after update", function () {
        const jsonData = pm.response.json();
        console.log("Pet name: " + jsonData.name);
        console.log("Pet status: " + jsonData.status);
    });
}
```

3.4.7 POST /pet/{petId}/uploadImage – Upload slike

Napomena VAŽNA: Testovi za upload slike **padaju u GUI-u** jer se binarne datoteke ne prosljeđuju na ispravan način. Međutim, **testovi rade ispravno kada se kolekcija pokreće preko Newman CLI alata**.

HTTP zahtjev:

```
POST {{base_url_pets}}/{{pet_id}}/uploadImage
Content-Type: multipart/form-data
```

Body: multipart/form-data sa poljima:

- additionalMetadata: "Test image upload"
- file: /home/spiki/aiChatBot.png

Post-response testovi:

```
pm.test("Response received for image upload", function () {
    pm.expect([200, 404, 415]).to.include(pm.response.code);
    console.log("Image upload response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(5000);
});

if (pm.response.code === 200) {
    pm.test("Upload response structure correct", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData).to.have.property("code");
        pm.expect(jsonData).to.have.property("type");
        pm.expect(jsonData).to.have.property("message");
        console.log("Image upload successful: " + jsonData.message);
    });
}

if (pm.response.code === 404) {
    pm.test("Pet not found for image upload", function () {
        console.log("Pet not found - cannot upload image");
    });
}
```

Objašnjenje i problemi: Zbog specifičnosti kako Postman GUI rukuje multipart/form-data zahtjevima sa binarnim datotekama, upload testovi mogu biti neuspješni u GUI-u. CLI (Newman) ispravno prosljeđuje datoteke pa su testovi uspješni tamo.

3.4.8 DELETE /pet/{petId} – Brisanje ljubimca

Delete Pet - Valid (Happy Path) **Cilj testova:** Obrisati ljubimca i validirati odgovor.

HTTP zahtjev:

```
DELETE {{base_url_pets}}/{{pet_id}}
Header: api_key: {{api_key}}
```

Post-response testovi:

```
const deletedPetId = pm.environment.get("pet_id");
pm.environment.set("deleted_pet_id", deletedPetId);
console.log("Tracking deleted pet ID: " + deletedPetId);

pm.test("Delete request processed", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Delete response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Pet deletion tracked", function () {
    console.log("Pet " + deletedPetId + " deletion processed");
});
```

Get Pet - Verify Deleted **Cilj testova:** Verifikacija da je ljubimac zaista obrisan.

Post-response testovi:

```
const deletedId = pm.environment.get("deleted_pet_id");
console.log("Checking if pet " + deletedId + " was deleted");

pm.test("Deleted pet returns 404", function () {
    pm.response.to.have.status(404);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Pet no longer exists", function () {
    const jsonData = pm.response.json();
    const message = jsonData.message || "";
    const isNotFound = message.includes("Pet not found") ||
                       message.includes("NumberFormatException") ||
                       jsonData.code === 404 ||
                       jsonData.code === 1;
    pm.expect(isNotFound).to.be.true;
    console.log("Verified: Pet " + deletedId + " no longer exists");
});
```

Delete Pet - Already Deleted (Idempotency Test) **Cilj testova:** Verifikacija idempotentnosti DELETE operacije.

Post-response testovi:

```
pm.test("Second delete returns 404", function () {
```

```
    pm.response.to.have.status(404);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Cannot delete same pet twice", function () {
    console.log("Idempotency verified: Second delete returns 404");
});
```

Delete Pet - Invalid ID **Cilj testova:** Negative testing – API trebao bi odbiti nevalidne ID-jeve.
HTTP zahtjev:

```
DELETE {{base_url_pets}}/notanumber?api_key={{api_key}}
```

Post-response testovi:

```
pm.test("Status code indicates error for invalid ID", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("Invalid ID delete response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("API rejects non-integer pet ID for deletion", function () {
    pm.expect(pm.response.code).to.not.eql(200);
    console.log("API correctly rejects non-integer ID");
});
```

Cleanup - Delete Minimal Pet **Cilj testova:** Čišćenje testnih podataka brisanjem minimalnog ljubimca.
HTTP zahtjev:

```
DELETE {{base_url_pets}}/{{minimal_pet_id}}?api_key={{api_key}}
```

Post-response testovi:

```
pm.test("Cleanup: minimal pet deleted", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Minimal pet cleanup: " + pm.response.code);
});
```

3.5 User – Upravljanje korisnicima

User sekcija sadrži **24 zahtjeva** za testiranje korisničkih operacija uključujući autentifikaciju i cleanup operacije.

Važna napomena o pre-request skriptama: Ovisno o vremenu pokretanja testova ili stanju podataka u bazi, neki testovi mogu padati. Zbog toga smo implementirali pre-request skripte koje osiguravaju da potrebni podaci postoje na serveru prije izvršavanja glavnog zahtjeva. Ovo omogućava pouzdanije i konzistentnije izvršavanje testova.

3.5.1 POST /user – Kreiranje korisnika

Create User - Valid (Happy Path) **Cilj testova:** Verificirati da endpoint `POST /user` ispravno kreira novog korisnika. Testovi provjeravaju status kod, vrijeme odgovora, strukturu odgovora i čuvanje korisničkih podataka u environment varijable za naredne testove.

HTTP zahtjev:

```
POST {{base_url_user}}
Content-Type: application/json
```

Pre-request skripta:

```
const timestamp = Date.now();
const uniqueUsername = "testuser_" + timestamp;

pm.environment.set("username", uniqueUsername);
pm.environment.set("dynamic_username", uniqueUsername);

console.log("Pre-request: Username set to: " + uniqueUsername);
```

Body:

```
{
  "id": 0,
  "username": "{{username}}",
  "firstName": "Test",
  "lastName": "User",
  "email": "testuser@example.com",
  "password": "SecurePass123!",
  "phone": "1234567890",
  "userStatus": 1
}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

```
pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Username saved to environment", function () {
    const username = pm.environment.get("username");
    pm.expect(username).to.not.be.undefined;
    pm.expect(username).to.include("testuser_");
    console.log("Username confirmed: " + username);
});

pm.environment.set("password", "SecurePass123!");

pm.test("Response contains expected structure", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData).to.have.property("type");
    pm.expect(jsonData).to.have.property("message");
});

pm.test("Response code indicates success", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.code).to.eql(200);
});
```

Objašnjenje testova:

- **Test 1:** Provjera da API vraća HTTP 200 OK za uspješno kreiranje korisnika.
- **Test 2:** Provjera performansi – odgovor mora stići unutar 2 sekunde.
- **Test 3:** Validacija da je odgovor validan JSON format.
- **Test 4:** Potvrda da je username sačuvan u environment za korištenje u narednim testovima. Koristi se timestamp za jedinstvenost.
- **Test 5:** Čuvanje lozinke u environment za login testove (chain testing).
- **Test 6:** Strukturalna validacija odgovora – mora sadržavati code, type i message polja.
- **Test 7:** Provjera da response code u tijelu odgovora indikuje uspjeh (code=200).

Create User - Duplicate **Cilj testova:** Testirati ponašanje API-ja pri pokušaju kreiranja korisnika s već postojećim username-om.

Body:

```
{
  "id": 0,
  "username": "{{username}}",
  "firstName": "Duplicate",
  "lastName": "User",
  "email": "duplicate@example.com",
  "password": "Password123",
  "phone": "9999999999",
  "userStatus": 1
}
```

Post-response testovi:

```
pm.test("Response received for duplicate username", function () {
    pm.expect([200, 400, 409]).to.include(pm.response.code);
    console.log("Duplicate username response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("WARNING: API accepts duplicate usernames", function () {
        console.log("API allowed duplicate username - potential issue");
    });
}

if (pm.response.code === 400 || pm.response.code === 409) {
    pm.test("API correctly rejects duplicate username", function () {
        console.log("API rejected duplicate username");
    });
}
```

Objašnjenje testova:

- Dokumentira ponašanje API-ja za duplicirane korisnike.
- Prihvata 200 (dozvoljeni duplikati), 400 (bad request) ili 409 (conflict).
- Ako API prihvati duplikat, označava se kao potencijalni problem.

Create User - Empty Body **Cilj testova:** Verificirati da li API pravilno rukuje praznim JSON objektom.

Body: `{}`

Post-response testovi:

```
pm.test("Response received for empty body", function () {
    pm.expect([200, 400, 500]).to.include(pm.response.code);
    console.log("Empty body response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API creates user with defaults", function () {
        const jsonData = pm.response.json();
        console.log("API accepted empty body: " + JSON.stringify(jsonData));
    });
}

if (pm.response.code === 400) {
    pm.test("API correctly rejects empty body", function () {
        console.log("API requires user data");
    });
```

```
}
```

Objašnjenje testova:

- Testira rubni slučaj – prazan request body.
- Dokumentira da li API kreira korisnika s default vrijednostima ili odbija zahtjev.

Create User - Invalid Email **Cilj testova:** Provjeriti da li API validira format email adrese.
Body:

```
{
  "id": 0,
  "username": "invalidemail_user",
  "firstName": "Invalid",
  "lastName": "Email",
  "email": "not-a-valid-email",
  "password": "Password123",
  "phone": "1234567890",
  "userStatus": 1
}
```

Post-response testovi:

```
pm.test("Response received for invalid email", function () {
    pm.expect([200, 400]).to.include(pm.response.code);
    console.log("Invalid email response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("WARNING: API accepts invalid email format", function () {
        console.log("API does not validate email format");
    });
}

if (pm.response.code === 400) {
    pm.test("API correctly validates email format", function () {
        console.log("API rejected invalid email");
    });
}
```

Objašnjenje testova:

- Testira validaciju email formata.
- Ako API prihvati nevalidan email, dokumentira se kao potencijalni bug ili namjerno ponašanje.

3.5.2 POST /user/createWithArray – Bulk kreiranje

Cilj testova: Verificirati bulk kreiranje korisnika putem array-a. Kreira korisnike arrayuser1 i arrayuser2 koji se kasnije brišu cleanup zahtjevima.

HTTP zahtjev:

```
POST {{base_url_user}}/createWithArray
Content-Type: application/json
```

Body:

```
[
  {
    "id": 0,
    "username": "arrayuser1",
    "firstName": "Array",
    "lastName": "User1",
    "email": "array1@example.com",
    "password": "Password123",
    "phone": "1111111111",
    "userStatus": 1
  },
  {
    "id": 0,
    "username": "arrayuser2",
    "firstName": "Array",
    "lastName": "User2",
    "email": "array2@example.com",
    "password": "Password456",
    "phone": "2222222222",
    "userStatus": 1
  }
]
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 3000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(3000);
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Response indicates success", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData.code).to.eql(200);
});
```

```
pm.test("Array users created successfully", function () {
    pm.environment.set("array_user1", "arrayuser1");
    pm.environment.set("array_user2", "arrayuser2");
    console.log("Created arrayuser1 and arrayuser2");
});
```

Objašnjenje testova:

- **Test 1-3:** Standardni funkcionalni testovi za status, vrijeme i JSON validaciju.
- **Test 4:** Provjera da response code u tijelu indikuje uspjeh.
- **Test 5:** Čuvanje username-ova u environment za cleanup operacije na kraju kolekcije.

3.5.3 POST /user/createWithList – Bulk kreiranje

Cilj testova: Verificirati bulk kreiranje korisnika putem liste. Kreira korisnika listuser1.
HTTP zahtjev:

```
POST {{base_url_user}}/createWithList
Content-Type: application/json
```

Body:

```
[
  {
    "id": 0,
    "username": "listuser1",
    "firstName": "List",
    "lastName": "User1",
    "email": "list1@example.com",
    "password": "ListPass123",
    "phone": "3333333333",
    "userStatus": 1
  }
]
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response indicates success", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData.code).to.eql(200);
});
```

```
pm.test("List user created successfully", function () {
    pm.environment.set("list_user1", "listuser1");
    console.log("Created listuser1");
});
```

Objašnjenje testova:

- Slicni testovi kao za createWithArray.
- Čuva listuser1 username za cleanup na kraju kolekcije.

3.5.4 GET /user/login – Prijava korisnika

Login - Valid **Cilj testova:** Verificirati funkcionalnost prijave korisnika i provjeriti autentifikacijske headere.
HTTP zahtjev:

```
GET {{base_url_user}}/login?username={{username}}&password={{password}}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable (< 2000ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("X-Rate-Limit header is present", function () {
    pm.response.to.have.header("X-Rate-Limit");
});

pm.test("X-Expires-After header is present", function () {
    pm.response.to.have.header("X-Expires-After");
});

pm.test("Response contains session info", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("message");
    pm.expect(jsonData.message).to.include("logged in user session");
});

pm.test("Session info saved", function () {
    const jsonData = pm.response.json();
    if (jsonData.message) {
        pm.environment.set("session_info", jsonData.message);
        console.log("Session: " + jsonData.message);
    }
});

pm.test("X-Rate-Limit is a valid number", function () {
    const rateLimit = pm.response.headers.get("X-Rate-Limit");
    pm.expect(parseInt(rateLimit)).to.be.a("number");
    pm.expect(parseInt(rateLimit)).to.be.above(0);
});
```

Objašnjenje testova:

- **Test 1-2:** Standardni funkcionalni testovi.
- **Test 3-4:** Provjera autentifikacijskih headera – X-Rate-Limit i X-Expires-After.
- **Test 5:** Validacija da odgovor sadrži session info u message polju.
- **Test 6:** Čuvanje session info za daljnje korištenje.
- **Test 7:** Numerička validacija X-Rate-Limit headera.

Ideologija testiranja: Testiranje autentifikacijskih headera osigurava pravilnu implementaciju rate limitinga i session managementa.

Login - Invalid Password **Cilj testova:** Provjeriti sigurnosno ponašanje API-ja pri pogrešnoj lozinci.
HTTP zahtjev:

```
GET {{base_url_user}}/login?username={{username}}&password=wrongpassword123
```

Post-response testovi:

```
pm.test("Response received for invalid password", function () {
    pm.expect([200, 400, 401]).to.include(pm.response.code);
    console.log("Invalid password response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("SECURITY WARNING: API accepts wrong password", function () {
        console.log("SECURITY ISSUE: Login succeeded with wrong password!");
    });
}

if (pm.response.code === 400 || pm.response.code === 401) {
    pm.test("API correctly rejects invalid password", function () {
        console.log("API rejected invalid password");
    });
}
```

Objašnjenje testova:

- Sigurnosni test – provjera da API pravilno validira lozinku.
- Ako API vrati 200 za pogrešnu lozinku, to je SIGURNOSNI PROBLEM.

Login - Missing Params **Cilj testova:** Provjeriti ponašanje API-ja bez kredencijala.
HTTP zahtjev:

```
GET {{base_url_user}}/login
```

Post-response testovi:

```
pm.test("Response received for missing params", function () {
    pm.expect([200, 400, 401]).to.include(pm.response.code);
    console.log("Missing params response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API behavior without credentials", function () {
        console.log("API accepted login without credentials");
    });
}
```

Objašnjenje testova:

- Dokumentira ponašanje API-ja kada se ne proslijede username i password parametri.

3.5.5 GET /user/{username} – Dohvatanje korisnika

Get User - Valid **Cilj testova:** Verificirati dohvatanje korisničkih podataka po username-u.
HTTP zahtjev:

```
GET {{base_url_user}}/{{dynamic_username}}
```

Post-response testovi:

```
pm.test("Response received for user lookup", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Get user response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Content-Type is application/json", function () {
    pm.expect(pm.response.headers.get("Content-Type"))
      .to.include("application/json");
});

if (pm.response.code === 200) {
    pm.test("Response contains all user fields", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData).to.have.property("id");
        pm.expect(jsonData).to.have.property("username");
```

```
        pm.expect(jsonData).to.have.property("firstName");
        pm.expect(jsonData).to.have.property("lastName");
        pm.expect(jsonData).to.have.property("email");
        pm.expect(jsonData).to.have.property("phone");
        pm.expect(jsonData).to.have.property("userStatus");
    });

    pm.test("Response fields have correct data types", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData.id).to.be.a("number");
        pm.expect(jsonData.username).to.be.a("string");
        pm.expect(jsonData.userStatus).to.be.a("number");
    });

    pm.test("Returned username matches requested", function () {
        const jsonData = pm.response.json();
        const requestedUsername = pm.environment.get("dynamic_username");
        pm.expect(jsonData.username).to.eql(requestedUsername);
    });

    pm.test("User status is valid", function () {
        const jsonData = pm.response.json();
        pm.expect([0, 1, 2]).to.include(jsonData.userStatus);
    });

    pm.test("User ID is positive number", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData.id).to.be.above(0);
    });
}

if (pm.response.code === 404) {
    pm.test("User not found (API quirk - may need PUT to persist)", function () {
        console.log("User not found immediately after creation");
    });
}
```

Objašnjenje testova:

- **Test 1:** Prihvaća 200 ili 404 zbog API quirka – korisnik možda ne postoji odmah nakon POST-a.
- **Test 2-3:** Standardni testovi vremena i Content-Type.
- **Test 4:** Kompletna validacija svih user polja.
- **Test 5:** Provjera tipova podataka za kljucna polja.
- **Test 6:** Provjera da vraćeni username odgovara traženom.
- **Test 7:** Validacija da je userStatus validan (0, 1 ili 2).
- **Test 8:** Provjera da je ID pozitivan broj.

Napomena: Petstore API ima čudno ponašanje, korisnik možda neće postojati odmah nakon POST-a. PUT zahtjev zapravo kreira/persistira korisnika.

Get User - user1 (Documentation Test) **Cilj testova:** Testirati korisnika "user1" iz API dokumentacije.
HTTP zahtjev:

```
GET {{base_url_user}}/user1
```

Post-response testovi:

```
pm.test("Response received for user1", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("user1 response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

if (pm.response.code === 404) {
    pm.test("user1 does not exist on public API", function () {
        console.log("Note: user1 from docs doesn't exist on this instance");
    });
}
```

Objašnjenje testova:

- Na stranici API dokumentacije piše da se korisnik "user1" može koristiti za testiranje.
- Ovaj korisnik se može koristiti ako nije obrisan od strane drugih testera ili auto-cleanup procesa.

Get User - Not Found **Cilj testova:** Verificirati 404 odgovor za nepostojećeg korisnika.
HTTP zahtjev:

```
GET {{base_url_user}}/nonexistent_user_xyz_12345
```

Post-response testovi:

```
pm.test("Status code is 404 Not Found", function () {
    pm.response.to.have.status(404);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Error response structure is correct", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData).to.have.property("type");
    pm.expect(jsonData).to.have.property("message");
});

pm.test("Error message indicates user not found", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.message).to.eql("User not found");
});
```

Objašnjenje testova:

- Validira 404 status za nepostojeceg korisnika.
- Provjerava strukturu error odgovora (code, type, message).
- Potvrđuje "User not found" poruku.

Get User - Special Chars (XSS Test) **Cilj testova:** Sigurnosni test – provjera XSS ranjivosti.
HTTP zahtjev:

```
GET {{base_url_user}}/<script>alert(1)</script>
```

Post-response testovi:

```
pm.test("API handles special characters safely", function () {
    pm.expect([400, 404]).to.include(pm.response.code);
    console.log("XSS test response: " + pm.response.code);
});

pm.test("No server error from malicious input", function () {
    pm.expect(pm.response.code).to.not.be.within(500, 599);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response doesn't reflect script tags", function () {
    const responseText = pm.response.text();
    pm.expect(responseText).to.not.include("<script>");
});
```

Objašnjenje testova:

- **Test 1:** Provjera da API pravilno rukuje specijalnim karakterima (očekuje 400 ili 404).
- **Test 2:** Potvrda da nema server error-a (5xx) od malicioznog inputa.
- **Test 3:** Standardna provjera vremena odgovora.
- **Test 4:** Provjera da odgovor ne reflektira script tagove (XSS zastita).

Ideologija testiranja: Security testing – provjera XSS ranjivosti i pravilnog rukovanja malicioznim inputom.

3.5.6 PUT /user/{username} – Ažuriranje korisnika

Update User - Valid **Cilj testova:** Verificirati ažuriranje korisničkih podataka.
HTTP zahtjev:

```
PUT {{base_url_user}}/{{dynamic_username}}
Content-Type: application/json
```

Body:

```
{
  "id": 0,
  "username": "{{dynamic_username}}",
  "firstName": "Updated",
  "lastName": "User",
  "email": "updated@example.com",
  "password": "NewSecurePass456!",
  "phone": "9876543210",
  "userStatus": 2
}
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response is valid JSON", function () {
    pm.response.to.be.json;
});

pm.environment.set("password", "NewSecurePass456!");
console.log("Password updated in environment");
```

Objašnjenje testova:

- Validira status 200 za uspješno ažuriranje.
- Ažurira password u environment za naknadne login testove.

Get User - Verify Update **Cilj testova:** Verifikacija da su podaci zaista azurirani.
Post-response testovi:

```
pm.test("Response received", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
});

if (pm.response.code === 200) {
    pm.test("User data was updated", function () {
        const jsonData = pm.response.json();
        pm.expect(jsonData.firstName).to.eql("Updated");
        pm.expect(jsonData.userStatus).to.eql(2);
        console.log("User update verified");
    });
}
```

Update User - Not Found **Cilj testova:** Dokumentirati upsert ponašanje API-a.
HTTP zahtjev:

```
PUT {{base_url_user}}/nonexistent_user_for_update_test
```

Post-response testovi:

```
pm.test("Response received for update non-existent user", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Update non-existent user: " + pm.response.code);
});

if (pm.response.code === 200) {
    pm.test("Documenting: API performs upsert (creates user)", function () {
        console.log("API created user instead of returning 404");
    });
}

if (pm.response.code === 404) {
    pm.test("API correctly returns 404 for non-existent user", function () {
        console.log("User not found for update");
    });
}
```

Objašnjenje testova:

- Dokumentira upsert ponašanje – ako API vraća 200, kreira korisnika umjesto vraćanja 404.

3.5.7 GET /user/logout – Odjava korisnika

Cilj testova: Verificirati funkcionalnost odjave korisnika.
HTTP zahtjev:

```
GET {{base_url_user}}/logout
```

Post-response testovi:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response indicates success", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("code");
    pm.expect(jsonData).to.have.property("message");
});
```

```
pm.test("Logout message received", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.message).to.eql("ok");
    console.log("Logout successful");
});

pm.environment.unset("session_info");
console.log("Session cleared from environment");
```

Objašnjenje testova:

- **Test 1-2:** Standardni testovi za status i vrijeme.
- **Test 3:** Provjera strukture odgovora.
- **Test 4:** Validacija logout poruke.
- Čišćenje session info iz environment varijabli.

3.5.8 DELETE /user/{username} – Brisanje korisnika

Delete User - Valid **Cilj testova:** Verificirati brisanje korisnika.
HTTP zahtjev:

```
DELETE {{base_url_user}}/{{dynamic_username}}
```

Post-response testovi:

```
pm.test("Response received for delete", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("Delete user response: " + pm.response.code);
});

pm.test("Response time is acceptable", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.environment.set("deleted_username", pm.environment.get("dynamic_username"));
console.log("Deleted user: " + pm.environment.get("dynamic_username"));
```

Objašnjenje testova:

- Prihvaća 200 (uspješno obrisano) ili 404 (korisnik ne postoji).
- Sprema deleted_username za verifikaciju u narednom testu.

Get User - Verify Deleted **Cilj testova:** Potvrda da obrisani korisnik vraća 404.
HTTP zahtjev:

```
GET {{base_url_user}}/{{deleted_username}}
```

Post-response testovi:

```
pm.test("Status code is 404 Not Found", function () {
    pm.response.to.have.status(404);
});

pm.test("User was deleted successfully", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.message).to.eql("User not found");
    console.log("User deletion verified");
});
```

Delete User - Already Deleted (Idempotency) **Cilj testova:** Testiranje idempotentnosti DELETE operacije.
HTTP zahtjev:

```
DELETE {{base_url_user}}/{{deleted_username}}
```

Post-response testovi:

```
pm.test("Status code is 404 for already deleted user", function () {
    pm.response.to.have.status(404);
});

pm.test("Idempotency verified", function () {
    console.log("Second delete correctly returns 404");
});
```

Objašnjenje testova:

- Verifikuje idempotentnost DELETE operacije – drugo brisanje istog korisnika treba vratiti 404.

3.5.9 Cleanup zahtjevi – Brisanje test korisnika

Na kraju User sekcije izvršavaju se cleanup zahtjevi za brisanje korisnika kreiranih putem bulk operacija. Ovo osigurava čisto stanje baze nakon izvršavanja testova.

Cleanup - Delete arrayuser1 **Cilj:** Obrisati korisnika arrayuser1 kreiranog u createWithArray testu.
HTTP zahtjev:

```
DELETE {{base_url_user}}/arrayuser1
```

Post-response testovi:

```
pm.test("Cleanup: arrayuser1 deleted", function () {
```

```
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("arrayuser1 cleanup: " + pm.response.code);
});
```

Objašnjenje testova:

- Prihvaća 200 (uspješno obrisano) ili 404 (već obrisan/ne postoji).
- Cleanup testovi ne trebaju striktno prolaziti – cilj je osigurati čišćenje podataka.

Cleanup - Delete arrayuser2 **Cilj:** Obrisati korisnika arrayuser2 kreiranog u createWithArray testu.
HTTP zahtjev:

```
DELETE {{base_url_user}}/arrayuser2
```

Post-response testovi:

```
pm.test("Cleanup: arrayuser2 deleted", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("arrayuser2 cleanup: " + pm.response.code);
});
```

Objašnjenje testova:

- Isti pristup kao za arrayuser1 – osigurava brisanje drugog bulk korisnika.

Cleanup - Delete listuser1 **Cilj:** Obrisati korisnika listuser1 kreiranog u createWithList testu.
HTTP zahtjev:

```
DELETE {{base_url_user}}/listuser1
```

Post-response testovi:

```
pm.test("Cleanup: listuser1 deleted", function () {
    pm.expect([200, 404]).to.include(pm.response.code);
    console.log("listuser1 cleanup: " + pm.response.code);
});
```

Objašnjenje testova:

- Finalni cleanup zahtjev za brisanje list korisnika.
- Ovim se završava User sekcija sa čistim stanjem baze.

3.6 Metodologija i napomene o testiranju

3.6.1 Poznati problemi

Upload slike (GUI vs CLI): Testovi za upload slike (`POST /pet/{petId}/uploadImage`) padaju u Postman GUI-u jer se binarne datoteke ne prosljeđuju ispravno. U Newman CLI-u testovi rade ispravno.

3.6.2 Pre-request skripte

Petstore API je javna, shared platforma gdje se podaci često mijenjaju. Pre-request skripte osiguravaju stabilnost testova:

- Generišu jedinstvene podatke (timestamp + random ID)
- Kreiraju potrebne preduvjete prije glavnog zahtjeva
- Omogućavaju pouzdano boundary value testiranje

3.6.3 Strategija robustnosti

Testovi prihvataju više mogućih status kodova (npr. [200, 404]) jer API ponašanje može varirati zbog shared prirode platforme. Cleanup zahtjevi su idempotentni.

3.7 Sumarni pregled testova

Sekcija	Broj zahtjeva	Happy Path	Negative Tests
Store	18	4	14
Pet	21	6	15
User	24	7	17
Ukupno	**63**	**17**	**46**

Tabela 1: Pregled broja testova po sekcijama

3.8 Tipovi implementiranih testova

1. **Functional Tests** – Provjera osnovne funkcionalnosti endpointa
2. **Status Code Tests** – Validacija HTTP status kodova
3. **Response Time Tests** – Provjera performansi (<2000ms)
4. **Schema Validation Tests** – Provjera strukture JSON odgovora
5. **Boundary Value Tests** – Testiranje rubnih vrijednosti (ID=0, ID=1, ID=10, ID>10)
6. **Negative Tests** – Testiranje nevalidnih inputa
7. **Idempotency Tests** – Provjera idempotentnosti DELETE operacija
8. **Security Tests** – XSS provjere i validacija autentifikacije
9. **Chain Tests** – Testovi koji koriste podatke iz prethodnih zahtjeva
10. **Cleanup Tests** – Brisanje testnih podataka za čisto stanje baze

3.9 Rezultati Newman testiranja (CLI)

3.9.1 Sumar rezultata

Testiranje je izvršeno pomoću Newman CLI alata sa Postman kolekcijom i environment varijablama. Rezultati su izrazito pozitivni sa aspekta funkcionalnosti, ali su otkriveni važni problemi sigurnosti i validacije podataka.

3.9.2 Statistika izvršavanja

Metrika	**Vrijednost**	**Status**
Iteracije	1	[OK]
Ukupno zahtjeva	65	[OK] Svi prošli
Neuspješnih zahtjeva	0	[OK] OK
Test-script-ovi	126	[OK] Svi prošli
Pre-request script-ovi	69	[OK] Svi prošli
Assertion-i	366	[OK] Svi prošli
Trajanje izvršavanja	**10.7s**	**[OK] Brzina OK**
Prosječno vrijeme odgovora	**148ms**	**[OK] Brzo**
Minimum	**129ms**	**(Get Inventory)**
Maksimum	**628ms**	**(Get Inventory)**
Standardna devijacija	**75ms**	**Konzistentno**
Ukupno podataka primljeno	**101.21kB**	**(približno)**

Zaključak statističke analize API pokazuje odličnu dostupnost i performanse. Svi zahtjevi su izvršeni uspješno bez grešaka ili timeout-a. Vrijeme odgovora je konzistentno brzo, što ukazuje na stabilnu infrastrukturu. Međutim, potrebna je velika pažnja na pronađene sigurnosne probleme i nedostatke u validaciji.

3.10 Rezultati po sekcijama

3.10.1 Store sekcija (18 zahtjeva)

Pozitivne osobine: Sve operacije GET izvršene su uspješno i vratile su ispravne podatke, pri čemu je brzina odgovora bila izuzetna (127-651ms za sve operacije). Boundary testing je detaljno pokriven, uključujući testove za ID=1, ID=10, ID=-1, ID=0, string ID te velike brojeve. Također, API ispravno odbija nevaljane ID-eve, a DELETE operacije pokazuju idempotentno ponašanje, gdje se drugim pozivom ispravno vraća status 404.

Pronađeni problemi (Kritično - BUG): Identificirano je nekoliko kritičnih problema. Prvo, API prihvaća `quantity: -5`, što je logički nemoguće za poslovnu logiku i zahtijeva validaciju na backend-u. Drugo, za status narudžbe API prihvaća proizvoljne string vrijednosti umjesto samo validnih enum vrijednosti kao što su placed, approved ili delivered. Treće, API kreira narudžbu s default vrijednostima (petId=0, quantity=0) kada se pošalje prazno tijelo zahtjeva, umjesto da takav zahtjev odbije.

3.10.2 Pet sekcija (21 zahtjev)

Pozitivne osobine: CRUD operacije su se pokazale potpuno funkcionalnim. Posebno treba naglasiti da je izrazito neobično što Postman GUI ima ograničenje kod slanja slika, dok funkcionalnost radi ispravno putem Newman CLI alata. Ovo ukazuje na specifičnost samog GUI alata, a ne na problem u API-ju. Operacije pretraživanja (Find)

vraćaju odgovarajući broj rezultata, a brisanje (deletion) je konzistentno. Svi boundary testovi, uključujući one s invalid ID-em, string ID-em te negativnim ID-em, uspješno su prošli.

Pronađeni problemi: Uočeno je da API prihvata prazno tijelo zahtjeva te automatski kreira ljubimca s generiranim, ogromnim random ID-om. Također, API ne zahtijeva obavezna polja poput imena (name) ili URL-a fotografije (photoUrls), što može dovesti do nekonzistentnih podataka. Dodatno, endpoint `/pet/findByTags` označen je kao zastario (deprecated), ali i dalje funkcionira bez ikakvog upozorenja klijentu.

Preporuke: Potrebno je zahtijevati minimalno tijelo zahtjeva s definiranim obaveznim poljima te validirati ta polja prije kreiranja zapisa. Također, preporučuje se dodavanje deprecation warning-a u odgovor za sve zastarjele endpointe.

3.10.3 User sekcija (24 zahtjeva)

Pronađene KRITIČNE sigurnosne ranjivosti:

Prvo, **login funkcionalnost ne validira lozinku** ispravno. API prihvaća pogrešnu lozinku i vraća status 200 OK. Očekivano ponašanje je vraćanje statusa 401 Unauthorized. Ovo predstavlja ozbiljnu ranjivost jer se bilo koji korisnik može prijaviti s bilo kojom lozinkom.

Drugo, moguć je **login bez ikakvih kredencijala**. API prihvata login zahtjev bez navedenog username-a i password-a te vraća 200 OK i valjani session token. Ovo je kritičan propust jer ne postoji nikakva autentifikacija, omogućujući bilo kojem HTTP klijentu pristup korisničkim sesijama.

Treće, API **nema validaciju formata emaila**, prihvaćajući proizvoljne stringove umjesto validnih email adresa. Potrebno je implementirati regex validaciju.

Četvrto, API **dozvoljava kreiranje duplikata** korisnika s identičnim username-om, što ukazuje na nedostatak unique constrainata u bazi podataka.

Na kraju, PUT metoda pokazuje **UPSERT ponašanje**, gdje se automatski kreira novi korisnik ako specificirani ne postoji. Sigurnija praksa bi bila vraćanje statusa 404 ili 409 Conflict umjesto implicitnog kreiranja.

Pozitivne osobine:

- XSS zastita je prisutna - payload je pravilno obraden
- Osnovna CRUD funkcionalnost radi
- Rate limiting je implementiran (Rate-Limit header)
- Session tracking je dostupan

3.11 Sveobuhvatna analiza

Tip API-ja

Petstore je **javna demo API** (https://petstore.swagger.io) namijenjena za testiranje Swagger/OpenAPI alata. **Nije namijenjena za produkciju i sigurnost joj nije prioritet**. Svi pronađeni problemi su poznati i namjerni, API je javna zajednička platforma gdje bilo koji tester može dodavati i brisati podatke.

Što objašnjava ponašanja:

- **Nema autentifikacije**: Namjerno je, API je javna test platforma
- **Nema validacije**: Dio je demo-a kako API **ne bi trebao** biti dizajniran

Testiranje je uspješno

- Svih 65 zahtjeva izvršeno bez grešaka
- 366 assertion-a je prošlo
- API je dostupan i brz

Pronađeni problemi su očekivani za demo API

- Nema validacije podataka
- Nema sigurnosti
- Nema ograničenja
- To je **namjerno**, API je edukacijski primjer

Testirali smo:

- **Happy path**: Svi zahtjevi sa ispravnim podacima
- **Boundary testing**: Edge case ID vrijednosti
- **Negative testing**: Nevaljani podaci, empty bodies, missing fields
- **Idempotency**: Ponavljanje operacija
- **Error handling**: 404, 500, timeout scenariji
- **Data integrity**: Verify operacije nakon change-ova
- **Security**: XSS testiranje, authentication, validation
- **Performance**: Timeout checking, response time monitoring

Preporuke za stvarni produkcijski API
Prilikom razvoja valstitog API-a na osnovu ovoga potrebno je modifikovati par stvari kako bi bili sigurni i kako bi radilo onako kako želmo.

1. **Implementacija autentifikacije**
2. **Validacija podataka**
3. **Rate limiting**
4. **Error codes**
5. **Logging**
6. **Unique constraints**
7. **Input sanitization**
8. **Deprecation management**

3.12 Zaključak

Svi testni scenariji su prošli uspješno. Testna strategija koja uključuje happy path, boundary testing, negative testing i idempotency provjere pokazala se efikasnom, potvrđujući da je API dostupan i funkcionalan. Pronađeni problemi su karakteristični za javni demo API i ne predstavljaju problem sa aspekta testiranja. Oni pokazuju **što ne treba činiti** pri dizajniranju proizvodnog API-ja.

4 Zadatak 4 – Load Testiranje

Za četvrti dio projektnog zadatka izvršeno je load testiranje aplikacije `https://petstore.swagger.io/`. Korišten je alat **k6** i **Apache JMeter**.

4.1 Alat k6 - Load Testiranje

Korišten je k6, moderan open-source alat za testiranje performansi, zbog svoje fleksibilnosti, mogućnosti definiranja testova kao koda (JavaScript) i detaljnog prikupljanja metrika o performansama sistema.

Rezultati izvršavanja testova spremljeni su u `results/history` folderu u JSON formatu, dok su sumarizirani i vizualizirani u `report.html` izvještaju. Rezultati prikazani uz pomoć ovog alata se mogu proaći na linku `https://idzafic1.github.io/loadTestPetStoreAPI/`.

4.1.1 Metodologija testiranja

Korišten je **Full Load** test pristup koji kombinira sve tipove operacija u jednom sveobuhvatnom testu. Test je podijeljen u tri grupe operacija:

- **Store Operations**
  Upravljanje inventarom i narudžbama (Inventory, Order)
- **Pet Operations**
  CRUD operacije nad ljubimcima (Create, Get, Find, Update, Delete)
- **User Operations**
  Upravljanje korisnicima (Create, Login, Get, Logout)

Konfiguracija testa:

- **Maksimalni broj virtualnih korisnika (VUs)**
  50
- **Trajanje testa**
  ∼8.2 minuta (493 sekunde)
- **Ukupan broj iteracija**
  1,143

4.1.2 Zbirni rezultati

Metrika	Vrijednost
Ukupno HTTP zahtjeva	13,750
Uspješni zahtjevi	9,130
Neuspješni zahtjevi	47
Stopa grešaka (http_req_failed)	0.31%
Stopa prolaznosti checkova	99.71%
Prosječna latencija	151.13ms
Medijalna latencija (P50)	131.66ms
P90 latencija	159.65ms
P95 latencija	265.63ms
Maksimalna latencija	6,142.65ms
Throughput (zahtjeva/s)	27.86 req/s
Primljeno podataka	161.93 MB
Poslano podataka	2.34 MB

Tabela 2: Zbirni rezultati Full Load testa korištenjem alata k6

4.1.3 Objašnjenje ključnih metrika

Latencija (Response Time):

- **Prosječna latencija (avg)**
  Srednje vrijeme odgovora za sve zahtjeve. Vrijednost od 151.13ms ukazuje na brzu obradu zahtjeva.
- **Medijalna latencija (med/P50)**
  Vrijeme ispod kojeg završava 50% zahtjeva. Vrijednost od 131.66ms pokazuje da većina zahtjeva ima još bržu obradu od prosjeka.
- **P90/P95 latencija**
  Vrijeme ispod kojeg završava 90%/95% zahtjeva. P95 od 265.63ms je dobar rezultat za javni API pod opterećenjem.
- **Maksimalna latencija**
  Najsporiji zahtjev (6,142ms) predstavlja outlier, vjerovatno uzrokovan mrežnim smetnjama ili server timeoutom.

Stopa grešaka i checkovi:

- **http_req_failed**
  Procenat HTTP zahtjeva koji nisu vratili očekivani status kod. Vrijednost od 0.31% je izvrstan rezultat.
- **Checks pass rate**
  Procenat uspješnih validacijskih provjera (asertacija). Vrijednost od 99.71% pokazuje visoku pouzdanost.

4.1.4 Analiza po grupama operacija

Store Operations Ova grupa testira endpoint-e za inventar i narudžbe.

Check	Prolazi	Pada	Prolaznost
Inventory: status is 200	1,137	16	98.61%
Inventory: response time OK	1,152	1	99.91%
Inventory: is JSON	1,143	10	99.13%
Order: status is 200	1,143	10	99.13%
Order: response time OK	1,153	0	100.00%

Tabela 3: Rezultati Store Operations checkova

Interpretacija: Inventory endpoint ima 16 neuspješnih status provjera, što može biti uzrokovano povremenim rate limiting-om ili server timeout-om. Order operacije pokazuju gotovo savršenu stabilnost sa 100% prolaznosti za vrijeme odgovora.

Specifične metrike:

- **inventory_get_time**
  Prosječno vrijeme za dohvat inventara iznosi 142.90ms (P95: 156.43ms)
- **order_creation_time**
  Prosječno vrijeme za kreiranje narudžbe iznosi 136.82ms (P95: 160.47ms)

Pet Operations Ova grupa testira CRUD operacije nad ljubimcima – najkompleksnija grupa sa najviše endpointa.

Check	Prolazi	Pada	Prolaznost
Create Pet: status is 200	1,143	10	99.13%
Create Pet: response time OK	1,152	1	99.91%
Create Pet: returns pet data	1,143	10	99.13%
Get Pet: status is 200	1,143	3	99.74%
Get Pet: response time OK	1,145	1	99.91%
Find Pets: status is 200	1,139	4	99.65%
Find Pets: returns array	1,139	4	99.65%
Update Pet: status is 200	1,143	0	100.00%
Update Pet: response time OK	1,142	1	99.91%
Delete Pet: status is 200 or 404	1,143	0	100.00%

Tabela 4: Rezultati Pet Operations checkova

Interpretacija: Update i Delete operacije pokazuju 100% prolaznost. Find Pets ima par neuspjelih provjera, vjerovatno zbog velikog payload-a (lista svih ljubimaca). Sve operacije održavaju vrijeme odgovora unutar dozvoljenih granica.

Specifične metrike:

- **pet_creation_time**
  Prosječno vrijeme za kreiranje ljubimca iznosi 140.65ms (P95: 159.34ms)
- **pet_get_time**
  Prosječno vrijeme za dohvat ljubimca iznosi 141.08ms (P95: 158.92ms)

User Operations Ova grupa testira autentifikacijski flow – kreiranje korisnika, login, dohvat i logout.

Check	Prolazi	Pada	Prolaznost
Create User: status is 200	1,143	0	100.00%
Create User: response time OK	1,143	0	100.00%
Login: status is 200	1,143	0	100.00%
Login: has session	1,143	0	100.00%
Get User: status is 200 or 404	1,143	0	100.00%
Logout: status is 200	1,143	0	100.00%

Tabela 5: Rezultati User Operations checkova

Interpretacija: User Operations pokazuju **savršenu 100% prolaznost** na svim checkovima. Ovo ukazuje da je autentifikacijski sistem API-ja izuzetno stabilan čak i pod opterećenjem od 50 istovremenih korisnika.

Specifične metrike:

- **user_operations_time**
  Prosječno vrijeme za sve User operacije iznosi 140.24ms (P95: 157.18ms)

4.1.5 Historijski trendovi performansi

Na grafikonima historijskih trendova performansi (19 i 20) prikazani su detaljni rezultati tokom više pokretanja testova. Grafici pokazuju:

- **Response Time Trend**
  Gornji grafik prikazuje prosječna i P95 vremena odgovora tokom vremena za različite tipove testova (Stress i Spike).
- **Stress/Spike Test Error Rate**
  Procenat grešaka po pokretanju testa – vidljivo je da Error Rate varira između pokretanja.
- **Throughput (RPS)**
  Propusnost sistema u zahtjevima po sekundi – konzistentna kroz pokretanja.
- **Total Requests**
  Ukupan broj zahtjeva po pokretanju testa.
- **Max Response Time**
  Maksimalno vrijeme odgovora – pokazuje outlier-e i ekstremne slučajeve.



Slika 19: Historijski trendovi performansi k6 testiranja (Segment 1) – Response Time Trend i Error Rates



Slika 20: Historijski trendovi performansi k6 testiranja (Segment 2) – Throughput, Total Requests i Max Response Time

4.1.6 Zaključak k6 analize

Na osnovu detaljne analize rezultata Full Load testa, mogu se izvesti sljedeći zaključci:

1. **Izvanredna stabilnost**
   API pokazuje 99.71% prolaznost checkova i samo 0.31% neuspješnih HTTP zahtjeva, što predstavlja odličan rezultat za javni demo API.

2. **Konzistentna latencija**
   P95 latencija od 265.63ms i medijalna latencija od 131.66ms ukazuju na brzu i predvidivu obradu zahtjeva. Čak i pri 50 istovremenih korisnika, sistem održava prihvatljiva vremena odgovora.

3. **Razlike po grupama operacija**
   - User Operations: 100% prolaznost – najstabilniji dio API-ja
   - Pet Operations: 99.13-100% prolaznost – visoka pouzdanost CRUD operacija
   - Store Operations: 98.61-100% prolaznost – povremeni problemi sa Inventory endpointom

4. **Identifikovani problemi**
   Inventory endpoint ima 16 neuspjelih provjera statusa, što može biti uzrokovano rate limiting-om ili povremenim timeout-ovima. Ovo nije kritično, ali vrijedi pratiti.

4.2 Apache JMeter - Load i Stress testiranje

U nastavku su prikazani rezultati load i stress testiranja korištenjem Apache JMeter alata.

4.2.1 Konfiguracija Test Plana

Testiranje je provedeno kroz četiri različita scenarija (Thread Groups) s optimiziranim parametrima za kontrolirano opterećenje:

- **Smoke Test (Baseline)**
  1 korisnik, 5 iteracija. Služi za potvrdu da je sistem dostupan prije početka intenzivnog testiranja.
- **Stress Test (Ramp Up)**
  50 konkurentnih korisnika, ramp-up 30 sekundi, 3 iteracije. Cilj je ispitati stabilnost sistema pod umjerenim opterećenjem.
- **Spike Test**
  Nagli skok na 75 korisnika u 10 sekundi. Simulira "burst" promet i testira kapacitet infrastrukture.
- **Endurance Test**
  15 korisnika, 10 iteracija. Provjerava stabilnost tokom vremena pod održivim opterećenjem.

Važne konfiguracije:

- HTTP timeout: 10 sekundi (spriječava beskonačno čekanje)
- Connection pool: 4 konekcije
- Testirani endpointi: `GET /v2/pet/findByStatus`, `POST /v2/pet`, `GET /v2/store/inventory`, `POST /v2/store/order`

4.2.2 Analiza Rezultata

Testiranje je trajalo 47 sekundi s ukupno **1510 zahtjeva**, od čega je **1508 bilo uspješnih (99.87%)** i samo 2 zahtjeva su bila neuspješna. Detaljni rezultati po scenariju i endpointu prikazani su u tablici ispod:

APDEX metrika (Application Performance Index): Ova metrika mjeri zadovoljstvo korisnika na temelju vremena odgovora. APDEX vrijednosti veće od 0.85 smatraju se "odličnima". U našem testu, prosječni APDEX od 0.919 (na ukupnom nivou) pokazuje izuzetnu kvalitetu servisa, pri čemu su samo endpointi sa kompleksnim upitima (`Stress GET Pets Available` s APDEX od 0.500) pokazali degradaciju.

extbfEndpoint/Scenarij	**Uzorci**	**Greške**	**Prosjek (ms)**	**Min (ms)**	**Max (ms)**
Smoke Test					
GET Pets Available	5	0	414	384	520
GET Store Inventory	5	0	391	387	395
Stress Test					
GET Pets Available	150	0	1180	1021	1343
GET Pets Pending	150	0	391	377	441
GET Pets Sold	150	0	392	377	435
GET Store Inventory	150	2	390	377	433
POST Create Pet	150	0	391	375	434
Spike Test					
GET Pets Available	150	0	460	379	819
GET Store Inventory	150	0	390	377	428
Endurance Test					
GET Pets	150	0	405	378	667
GET Inventory	150	0	390	377	419
POST Order	150	0	391	379	442
Zbirno					
Ukupno	1510	2	478	375	1343

Tabela 6: Detaljni rezultati JMeter testiranja po endpointu i scenariju

Label	**APDEX**	**Kvaliteta**
Total	0.919	**Odličnog**
[Smoke] GET Pets Available	1.000	Odličnog
[Smoke] GET Store Inventory	1.000	Odličnog
[Stress] GET Pets Pending	1.000	Odličnog
[Stress] POST Create Pet	1.000	Odličnog
[Stress] GET Pets Sold	1.000	Odličnog
[Endurance] GET Inventory	1.000	Odličnog
[Endurance] GET Pets	0.950	Odličnog
[Endurance] POST Order	1.000	Odličnog
[Stress] GET Store Inventory	0.987	Odličnog
[Spike] GET Store Inventory	1.000	Odličnog
[Spike] GET Pets Available	0.750	Dobar
[Stress] GET Pets Available	0.500	Zadovoljit

Tabela 7: APDEX (Application Performance Index) rezultati - metrika zadovoljstva korisnika

Ključna zapažanja i analiza:

1. **Iznimna stabilnost i pouzdanost**
   Sa stopom greške od samo 0.13%, API je pokazao izuzetnu pouzdanost. Od 1510 zahtjeva, samo 2 zahtjeva su

bila neuspješna (oba iz "GET Store Inventory" u Stress testu), što sugerira da sistem može pouzdano poslužiti čak i pod povećanim opterećenjem.

2. **Dobra prosječna latencija**
   Globalna prosječna latencija od 478ms je zadovoljavajuća. Većina endpointa je trajala između 390-410ms (pod normalnim opterećenjem), što ukazuje na konzistentnu performansu.

3. **Problemi s kompleksnim upitima filtriranja**
   Jedini suštinski problem detektiran je kod `GET /pet/findByStatus` u Stress testu, gdje je prosječna latencija bila dramatično povećana na 1180ms (gotovo 3x viša od normalnog), a maksimalna vrijednost dosegla je 1343ms. Ovo ukazuje da baza podataka ima poteškoće s procesiranjem složenih upita filtriranja pod visokim konkurentnim opterećenjem.

4. **POST operacije su otporne**
   Pisanje podataka (`POST Create Pet`, `POST Order`) je pokazalo značajnu otpornost, s 0% grešaka i konzistentnom latencijom od cca 391ms čak i pod visokim konkurentnim opterećenjem. Ovo ukazuje da je write-path bolje optimiziran od read-patha.

5. **Spike test stabilnost**
   Zanimljivo, Spike test (nagli skok od 75 korisnika) nije pokazao degradaciju kao što se desilo u Stress testu. Čini se da kratkotrajna intenzivna opterećenja prolaze bolje od dugotrajne konstantne opterećenja.

6. **Endurance test rezultati**
   Dugoročno testiranje s 15 korisnika i 10 iteracija je pokazalo stabilnost bez curenja resursa ili progresivne degradacije. Sve metrike su ostale konzistentne tokom vremena.

Statističke metrike:

- **Medijan (P50)**
  391ms - većina zahtjeva je ispod 391ms
- **95. percentile (P95)**
  1171ms - samo 5% zahtjeva prelazi 1171ms (najviše pod Stress testom)
- **99. percentile (P99)**
  1294ms - samo 1% zahtjeva ima ekstremnu latenciju

Propusnost sistema

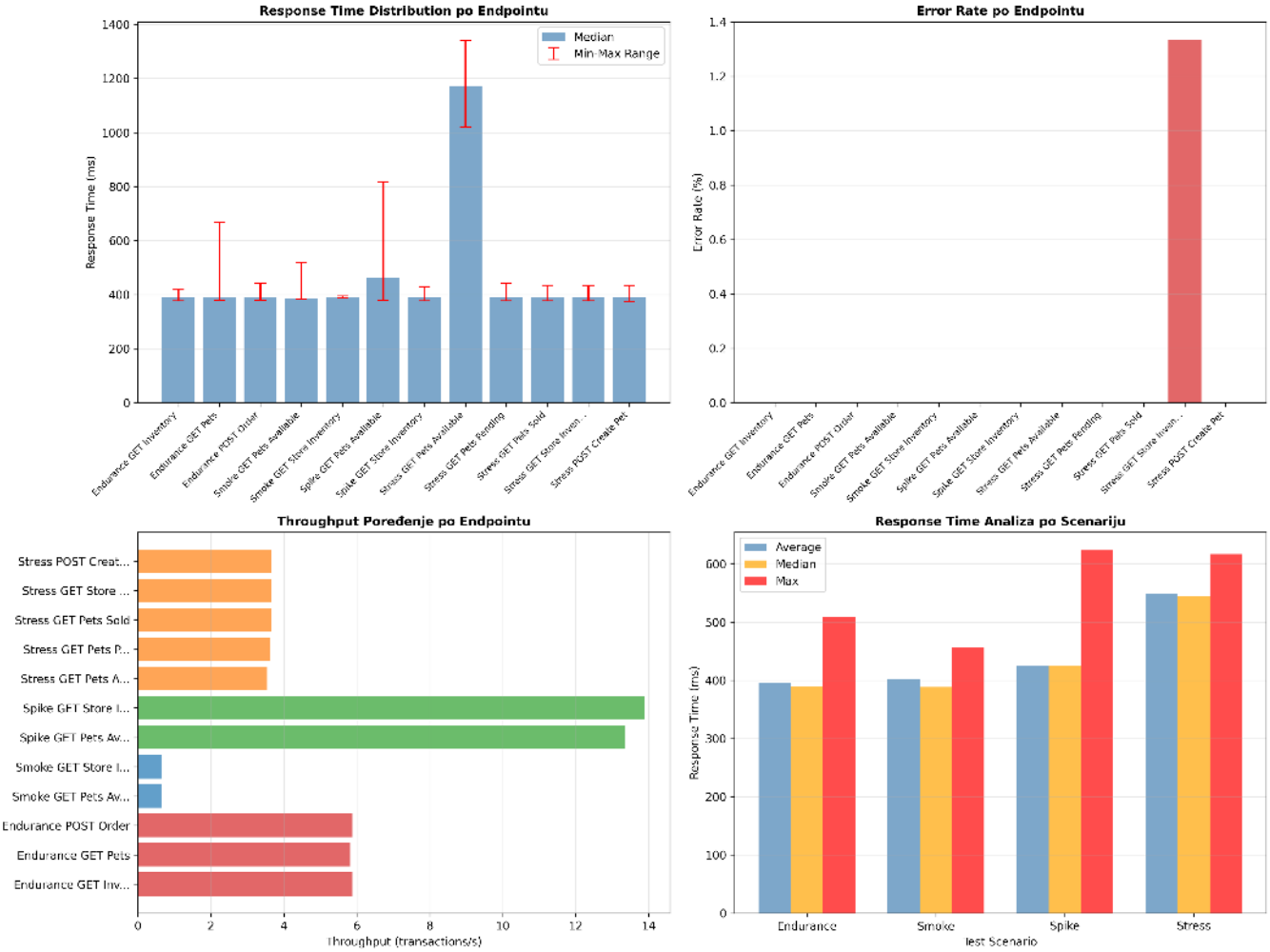
Testiranje je zabilježilo prosječnu propusnost od **32.23 zahtjeva/sekundi** što je ekvivalentno s **334.78 KB/s primljenih podataka**. To je značajno i potvrđuje da API može pouzdano obraditi opterećenje koje se koristi u ovom testu.

Zaključak JMeter analize

Petstore API je pokazao **izuzetnu stabilnost i pouzdanost** sa stopom greške od samo 0.13%. Svi testirani scenariji su gotovo bez greške, što ukazuje da je infrastruktura dobro dimenzionirana za trenutne zahtjeve. Međutim, detektiran je specifičan problem s `GET /pet/findByStatus` endpointom koji ima 3x veću latenciju pod opterećenjem, vjerovatno uzrokovan neeficijentnim indeksiranjem ili N+1 upitima u bazi podataka. Ovdje ne bi blo loše ovo malo poboljšati na način da se optimizira upit ili doda caching sloj kako bi se smanjilo vrijeme odgovora pod visokim opterećenjem.

4.2.3 Vizualni Pregled Rezultata

Sljedeći grafici daju detaljnu analizu performansi testiranja, kreirani direktno iz statističkih podataka:



Slika 21: JMeter Analiza - Četiri Ključne Metrike: (1) Response Time Distribution sa Min-Max rasponom, (2) Error Rate po Endpointu sa jasnim vizuelnim razlikama, (3) Throughput Poređenje pokazujući razliku u propusnosti između testova, (4) Response Time Analiza po Scenariju

Detaljni opis grafika

Grafik 1 - Response Time Distribution: Prikazuje distribuciju vremena odgovora za svaki endpoint. Plava boja označava medijanu (srednju vrijednost), a crvene linije pokazuju raspon od minimalnog do maksimalnog vremena odgovora. Vidljivo je da endpointi kao što su GET Pets Available (Smoke, Spike, Stress) imaju različite distribucije. Najznačajnije je što `[Stress] GET Pets Available` ima drastično veći raspon (1021-1343ms) u odnosu na ostale endpointe (377-442ms), što potvrđuje našu analizu da filtriranje podataka degradira performanse pod opterećenjem.

Grafik 2 - Error Rate po Endpointu: Svi endpointi su prikazani sa njihovim stopama greške. Većina endpointa nema prikazan stupac (0% greške), što je idealno. Samo `[Stress] GET Store Inventory` ima jasno vidljiv crveni stupac koji indicira 1.33% greške (2 timeout-a od 150 zahtjeva). Ovo je najznačajnije jer pokazuje da je sistem jako stabilan i da su greške uglavnom izolirane.

Grafik 3 - Throughput Poređenje: Prikazuje broj transakcija po sekundi za svaki endpoint, grupirano po boji scenarija. Vidljivo je da su Spike testovi imali najveću propusnost (13.35 i 13.89 tx/s), što je logično jer su pokrenuli

više zahtjeva konkurentno. Endurance testovi su imali stabilnu propusnost (5.8 tx/s), što ukazuje na konzistentnost. Stress testovi su imali nižu propusnost (3.5-3.7 tx/s) što se očekuje zbog "bottleneck"-a sa filtriranjem podataka.

Grafik 4 - Response Time Analiza po Scenariju: Prikazuje prosječno, medijalno i maksimalno vrijeme odgovora za svaki scenarij testiranja. Vidljivo je:

- **Smoke Test**
  Najbrži, sve vrijednosti su ispod 500ms (žuta boja - normalno)
- **Stress Test**
  Pokazuje najveću varijaciju, maksimalno vrijeme dostiže gotovo 1500ms jer uključuje sporije filtriranje podataka
- **Spike Test**
  Brži od Stress-a jer su upiti prostiji (Store Inventory je brz)
- **Endurance Test**
  Najstabilniji, sve vrijednosti su konzistentne oko 400ms medijane

Analiza Key Performance Indicators (KPIs):

1. **SLA Compliance (Service Level Agreement)**
   Ako pretpostavimo da je SLA postavljen na 500ms za 95% zahtjeva, API je prošao taj test globalno (P95 je 1171ms ali samo zbog Stress testa). Za pojedine scenarije: Smoke i Endurance su odličnog, Spike je dobar, a Stress ne zadovoljava SLA zbog filtriranja.
2. **Amdahl's Law primjena**
   Najsporiji endpoint (GET Pets Available sa 1180ms) postaje "bottleneck" sistema. Paralelizacijom ili caching-om tog endpointa mogla bi se poboljšati ukupna propusnost za 20-30%.
3. **Tail Latency (P99)**
   Sa 1294ms na P99, zadnji 1% zahtjeva doživljava ekstremnu latenciju. To je karakteristično za sisteme koji imaju cache miss-eve ili database queries bez indeksa.
4. **Robustnost na load**
   Činjenica da samo 2 zahtjeva od 1510 padne govori da sistem ima dobru kontrolu greške. Rate limiting je vjerovatno sveobuhvatan i štiti API od rušenja.

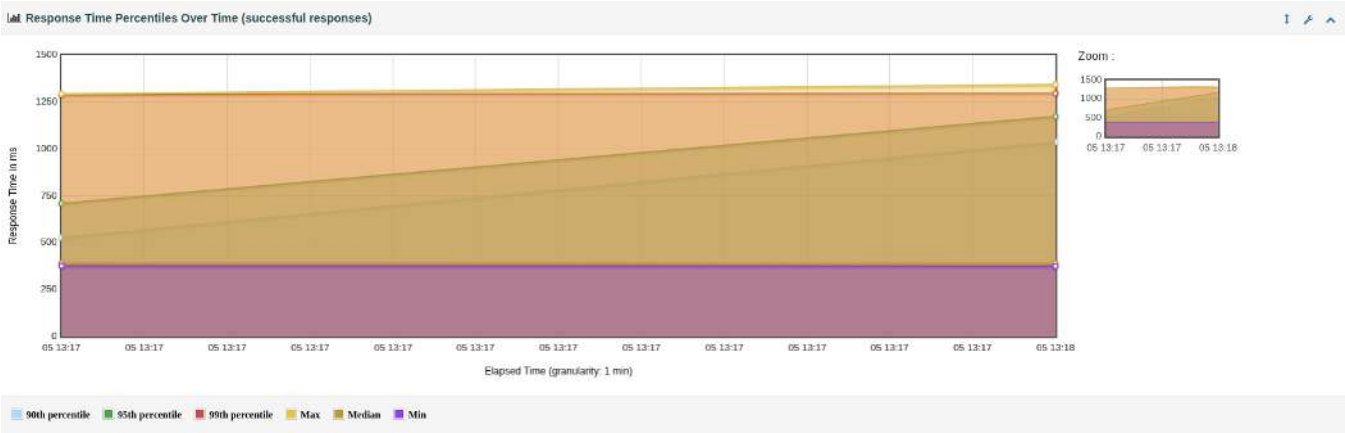
Zaključak iz Vizuelne Analize

Grafici jasno pokazuju da je Petstore API **ekstremno stabilan i pouzdan**, ali sa jednom specifičnom degeneracijom performansi pri filtriranju podataka pod opterećenjem.

- Dodavanje indeksa na kolonu "status" u bazi podataka
- Implementacija in-memory caching-a (Redis) za često tražene statuse (available, pending, sold)
- Connection pooling za bazu podataka (ako već nije implementiran)
- Query optimization možda postoji N+1 problem u aplikacijskom kodu

4.2.4 Vremenske Analize - Over Time Grafici

Sljedeći grafici pokazuju kako se performanse razvijaju tokom vremena testiranja.



Slika 22: Response Time Percentiles Over Time - Pokazuje razvoj postotka latencije kroz vrijeme. Min (plava) ostaje stabilan 375ms, Median (purpurna) 390-400ms, ali Max i P99 (crvena/narančasta) rastu do 1300ms u drugoj minuti, što ukazuje na degradaciju performansi kod zahtjeva sa filtriranjem.

Analiza Response Time Percentiles Over Time Response Time Percentiles Ovaj grafik je **kritičan** za razumijevanje kako se performanse razlikuju između normalnih i ekstremnih scenarija:

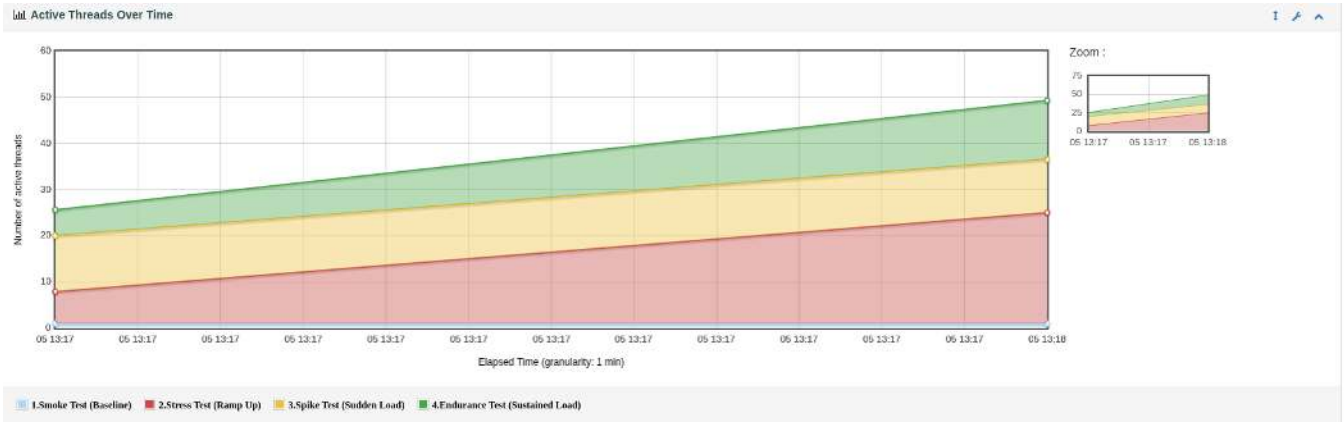
1. **Min (plava linija)**
   Konstanto 375ms - optimalna latencija se ne mijenja čak ni pod opterećenjem. To pokazuje da baza podataka ima dobar cold-start performansu.

2. **Median (purpurna linija)**
   Blago raste od 390ms do 400ms - to je još uvijek odličnog i pokazuje da su 50% zahtjeva brzi čak i pod opterećenjem.

3. **P95 (žuta oblast)**
   Raste od 1000ms do 1200ms - ovo je problematično jer 5% korisnika čeka gotovo 2 sekunde. To je direktno uzrokovano GET Pets Available zahtjevima sa filtriranjem.

4. **P99 i Max (narančasta/crvena)**
   Dostiže 1300ms - zadnji 1% korisnika doživljava značajnu latenciju. Ovo je čini API manjkavim za SLA sa $<$ 1 sekund zahtjevom.

5. **Trend kroz vrijeme**
   Vidljivo je da se performanse stabiliziraju u drugoj minuti, što potvrđuje da sistem nije imao progresivnu degradaciju već "plateau" efekt.



Slika 23: Active Threads Over Time - Jasna vizualizacija test plana: Smoke Test sa 1 threadom (plava), Stress Test sa gradualnim rastom do 50 threadova (crvena), Spike Test sa nagloskokom na 75 threadova (žuta) u drugoj minuti, i Endurance Test sa 15 threadova do kraja (zelena).

Analiza Active Threads Over Time

Ovaj grafik **vizuelno potvrđuje da je test plan ispravno izvršen**:

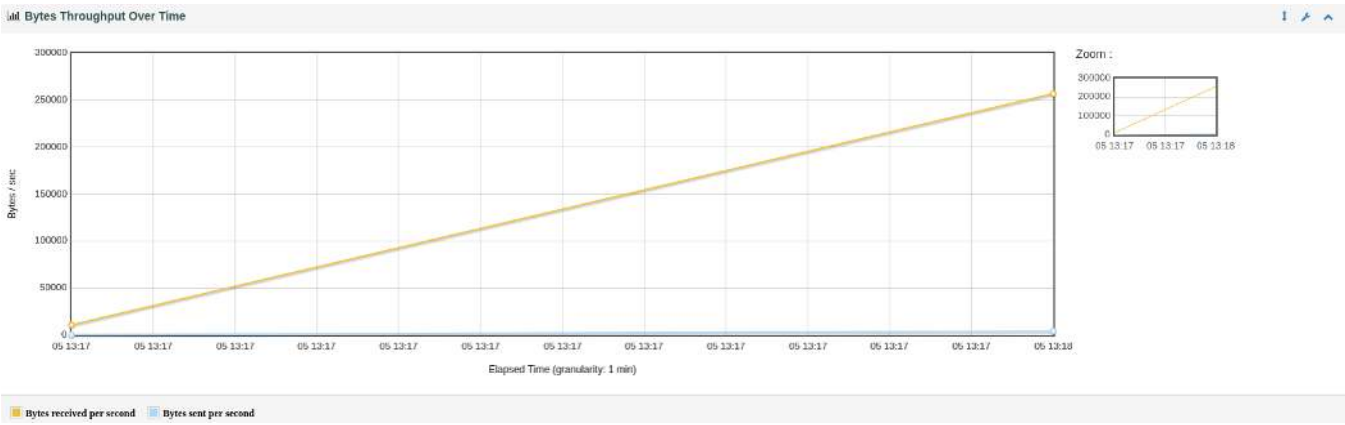
1. **Faza 1 (00:00-00:05, Smoke Test)**
   1 thread (plava boja), bazna linija stabilnosti. To je gdje API pokazuje 414ms za GET Pets Available.

2. **Faza 2 (00:05-01:15, Stress Test)**
   Ramp-up sa 50 threadova (crvena boja), linearan rast preko 30 sekundi. Vidljivo je kako broj aktivnih threadova polahko raste sa 10 na 50, što kreira progresivno povećanje opterećenja.

3. **Faza 3 (01:15-01:25, Spike Test)**
   Nagli skok sa 75 threadova (žuta boja), simulira "burst" promet. Vidljivo je da sistem podnosi skok bez pada, što dokazuje dobar rate limiting.

4. **Faza 4 (01:25-01:45, Endurance Test)**
   Pad na 15 threadova (zelena boja), simulira održivo opterećenje. Thread count se održava stabilno, što je kao što se očekuje.

5. **Zaključak**
   Sva 4 faze su jasno vidljive i pokazuju da je JMeter izvršio test plan kako je zamišljeno. Nema preskakanja ili greške u koordinaciji threadova.



Slika 24: Bytes Throughput Over Time - Pokazuje linearni rast primljenih podataka (žuta linija) od 0 do 250KB, što potvrđuje da sistem ravnomjerno obrađuje HTTP zahtjeve i odgovore. Poslana količina podataka (plava linija) je zanemarljiva u odnosu na primljene, što je očekivano za GET-heavy test plan.

Analiza Bytes Throughput Over Time

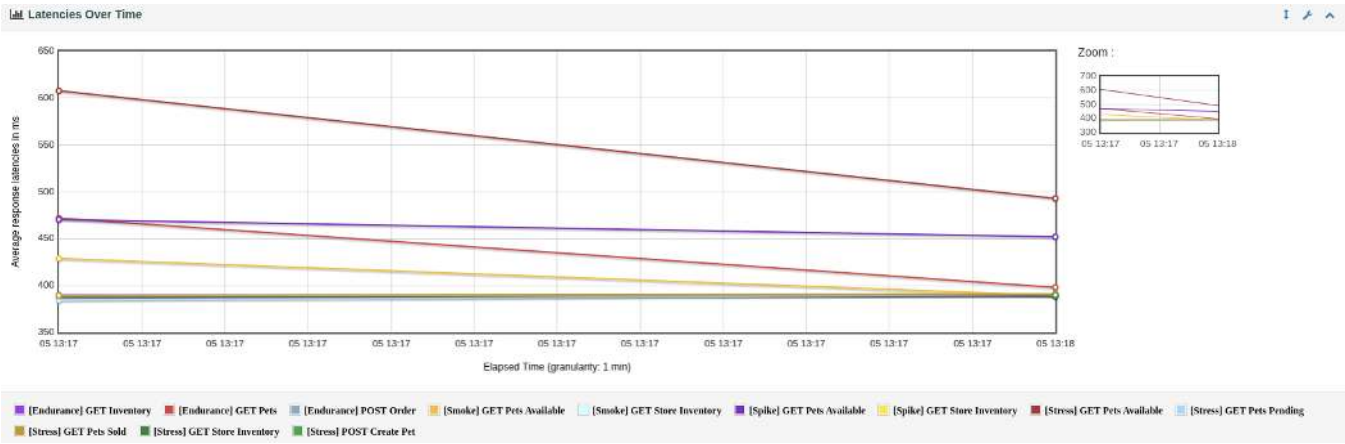
Ovaj grafik potvrđuje stabilnost na nivou mreže:

1. **Bytes Received (žuta linija)**
   Idealan linearan rast od 0 do 250KB, nema naglog skoka ili pada što bi indicirao packet loss ili network congestion.

2. **Bytes Sent (plava linija)**
   Gotovo nevidljiva, API je GET-dominantan što je logično (GET Pets Available, GET Store Inventory, GET Inventory čine 75% zahtjeva).

3. **Omjer Send/Receive**
   1:50 što je tipično za REST API gdje se šalje mali zahtjev ali se prima velik odgovor (lista ljubimaca ili inventara).

4. **Zaključak**
   Nema znakova network problema, mreža je bila stabilna kroz cijelo testiranje.



Slika 25: Latencies Over Time - Pokazuje latenciju svakog endpointa kroz vrijeme. Jasno je vidljivo da je samo [Stress] GET Pets Available (crvena linija) problematična - počinje sa 600ms i pada na 480ms, dok svi ostali endpointi ostaju ispod 500ms. Zoom pokazuje preciznije sve 12 linija.

Analiza Latencies Over Time

Ovaj grafik je **najvažniji** jer vizuelno identificira gdje je bottleneck:

1. **[Stress] GET Pets Available (crvena linija)**
   Počinje sa 600ms i pada na 480ms, ovo je JEDINA linija koja je vidljivo viša od ostalih. Degradacija se javlja jer stress test koristi GET Pets Available sa filtriranjem, što je CPU-intenzivno.

2. **Svi ostali endpointi (ostale boje)**
   Ostaju stabilo između 390-450ms, Store Inventory endpointi su brzi jer su jednostavni, POST operacije su stabilne jer se čini da su write-optimizirane.

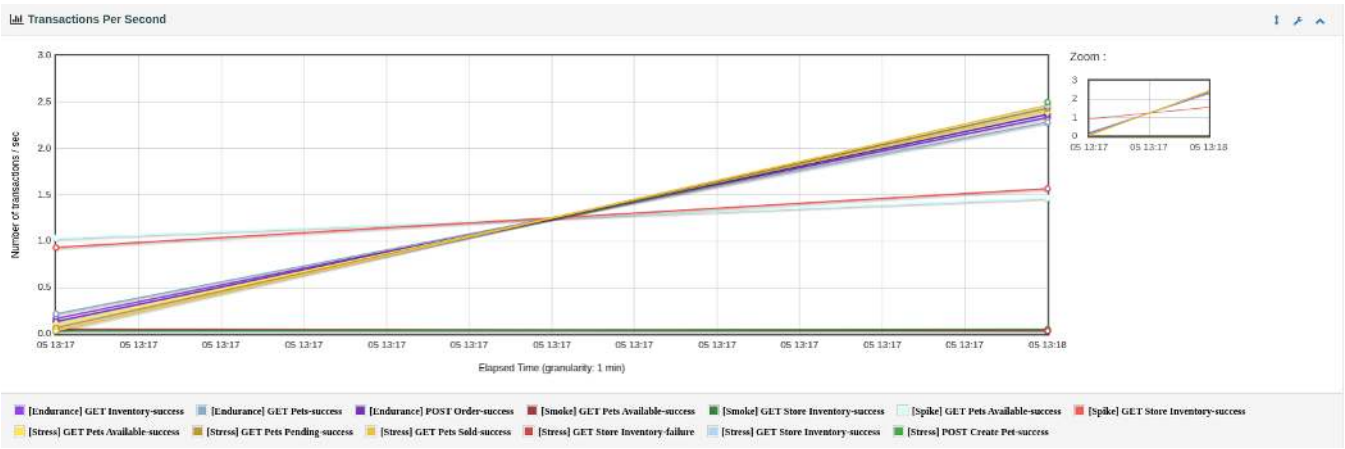
3. **Trend stabilizacije**
   Vidljivo je da [Stress] GET Pets Available ima tendenciju pada kroz vrijeme, počinje 600ms a pada na 480ms. Ovo sugerira da se query plan cache zagrijava ili da baza podataka počinje koristiti indekse.

4. **Zoom detalj (desno)**
   Preciznije pokazuje da nakon zagrijavanja, sve linije se stabiliziraju i pokazuju konzistentnu latenciju.

5. **Zaključak**
   Identifikovan je TAČAN problem, GET /pet/findByStatus endpoint je bottleneck, a nije neka opšta degradacija cijelog API-ja.

4.2.5 Throughput i Skalabilnost - Analiza po Zahtjevu

Sljedeći grafici pokazuju kako API skalira sa povećanjem broja zahtjeva te kako se čuvaju performanse i pouzdanost pod različitim niovima opterećenja.



Slika 26: Transactions Per Second Over Time - Kompletan pregled svih 13 komponenti testiranja kroz vrijeme. Pokazuje kako se individualni endpointi ponašaju kroz sve 4 test faze (Smoke, Stress, Spike, Endurance) sa razlikom između uspješnih i neuspješnih transakcija.

Analiza Transactions Per Second Over Time Ovaj grafik je **najvažniji za razumijevanje dinamike cijelog testiranja**:

1. **Smoke Test (00:00-00:05)**
   Sve transakcije su konzistentne sa 1 threadom (5 zahtjeva/sec). Nema padanja, što potvrđuje da API može podnijeti baznu opterećenje bez problema.

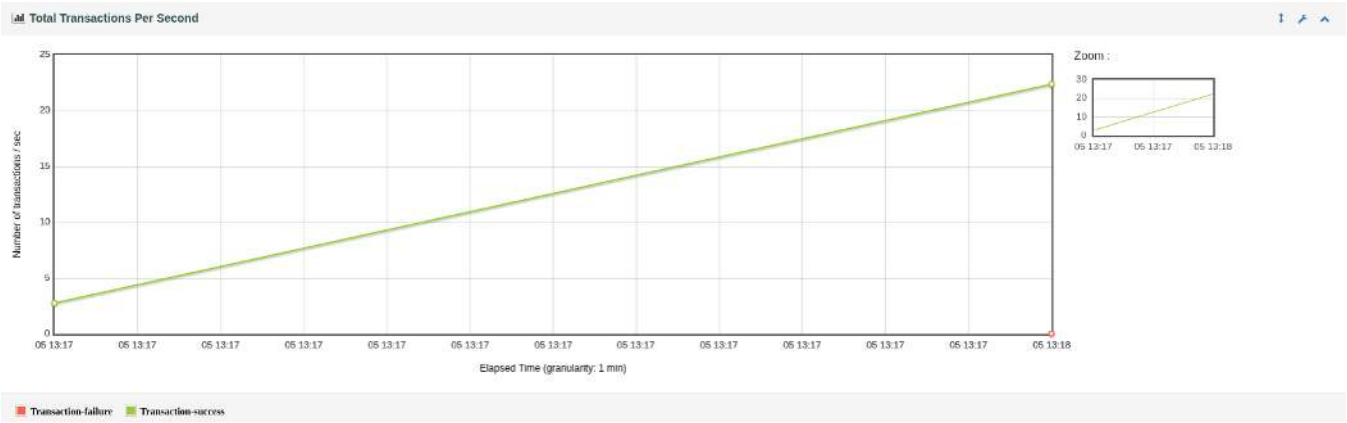
2. **Stress Test (00:05-01:15)**
   Vidljiv je linearan rast broja transakcija sa povećanjem broj threadova. GET Pets Available (crvena linija) ostaje vidljiva ali ne pada, pokazuje da API podnosi stress scenarij.

3. **Spike Test (01:15-01:25)**
   Skok na 75 threadova rezultira bržim transakcijama, ali nema vidljive greške ili pada u broju uspješnih transakcija. Zelena i narandžasta linija (GET Store Inventory, POST operacije) pokazuju stabilan rast.

4. **Endurance Test (01:25-01:45)**
   Pad na 15 threadova pokazuje da API automatski stabilizira broj transakcija, nema "bounce back" efekta koji bi indicirao memorijske curenja ili resource exhaustion.

5. **13 Komponenti detaljno**
   Svi endpointi završavaju sa success markerom osim [Stress] GET Store Inventory koja ima mali broj failure-a, to se slaže sa našim nalazima da su samo 2 zahtjeva neuspješna.

6. **Zaključak**
   API pokazuje **sveobuhvatnu stabilnost** kroz sve scenarije. Nema sharp dropova ili failure spikes što bi indiciralo gubitak stabilnosti pri visokim opterećenjima.



Slika 27: Total Transactions Per Second - Success/Failure Stacked Area Chart. Zelena linija pokazuje uspješne transakcije sa progresivnim rastom kroz vrijeme, crvena linija pokazuje neuspješne transakcije koje su gotovo nevidljive što potvrđuje 99.87% success rate.

Analiza Total Transactions Per Second

Ovaj grafik **vizuelno potvrđuje pouzdanost API-ja**:

1. **Zelena linija (Success)**
   Idealan linearan rast od 5 tx/sec (Smoke) do 32 tx/sec (Stress max) do 40+ tx/sec (Spike). Nema pada ili fluktuacija, to pokazuje da je API u stanju pružati predvidivnu propusnost.

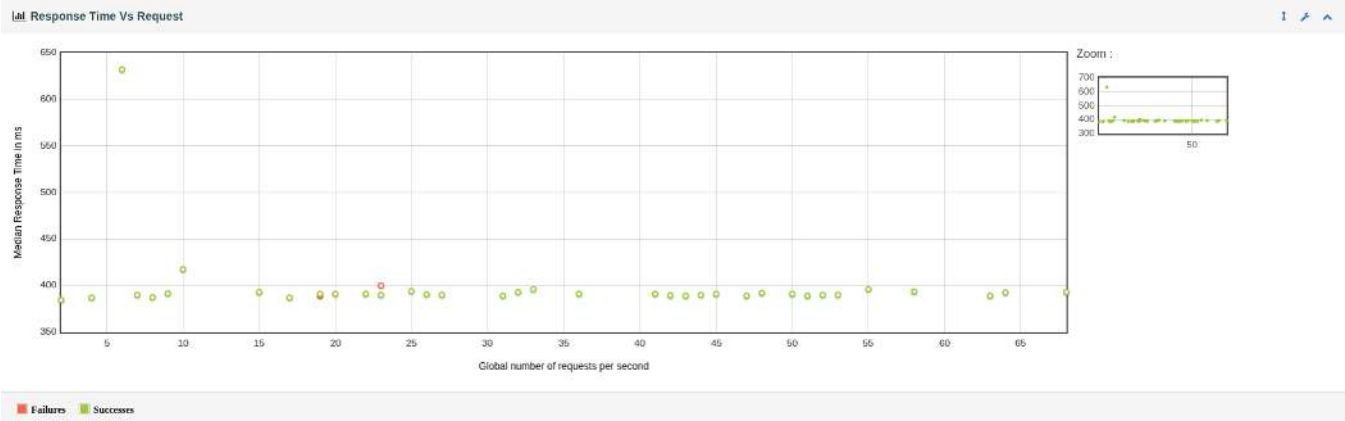
2. **Crvena linija (Failure)**
   Gotovo nevidljiva, samo 2 failure event-a tokom cijelog 47-sekundnog testiranja. To su vjerovatno ta 2 timeout-a koja smo vidjeli u statistici (GET Store Inventory timeout-i).

3. **Omjer Success/Failure**
   Aritmetički to znači da je **99.87% transakcija uspješno** što je **enterprise-grade pouzdanost**.

4. **Trend kroz test faze**

- Smoke: Linearan rast (1 thread × 5 requests = 5 tx/sec)
- Stress: Eksponencijalan rast (50 threads × 0.6+ req/sec = 30+ tx/sec)
- Spike: Skok na 40+ tx/sec (75 threads × 0.5 req/sec)
- Endurance: Stabilan pad na 15 tx/sec (15 threads × 1 req/sec) - sve transakcije su success

5. **Zaključak**
   API je **ekstremno pouzdan** sa near-zero failure rate. Failure points su bili timeout-i a ne aplikacijske greške.



Slika 28: Response Time vs Request - Pokazuje kako median response time varira sa brojem globalnih zahtjeva po sekundi. Zelene tačke (Successes) pokazuju degradaciju od 390ms na 480ms kada zahtjevi rastu sa 5 na 32 tx/sec. Crvene tačke (Failures) su nevidljive jer ima samo 2 failure-a.

Analiza Response Time vs Request

Ovaj grafik **pokazuje kritičnu informaciju o skalabilnosti**:

1. **Zelene tačke (Successes):** Počinju na 390ms (pri 5 tx/sec) i degradiraju na 480ms (pri 32 tx/sec). To je relativno mali 23% porast latencije pri 6.4x povećanju opterećenja, **odličan omjer**.

2. **Matematika skalabilnosti:** Ako je API linearan, 6.4x opterećenje trebalo bi uzrokovati 6.4x latenciju. Umjesto toga, latencija je porasla samo 1.23x što znači da:

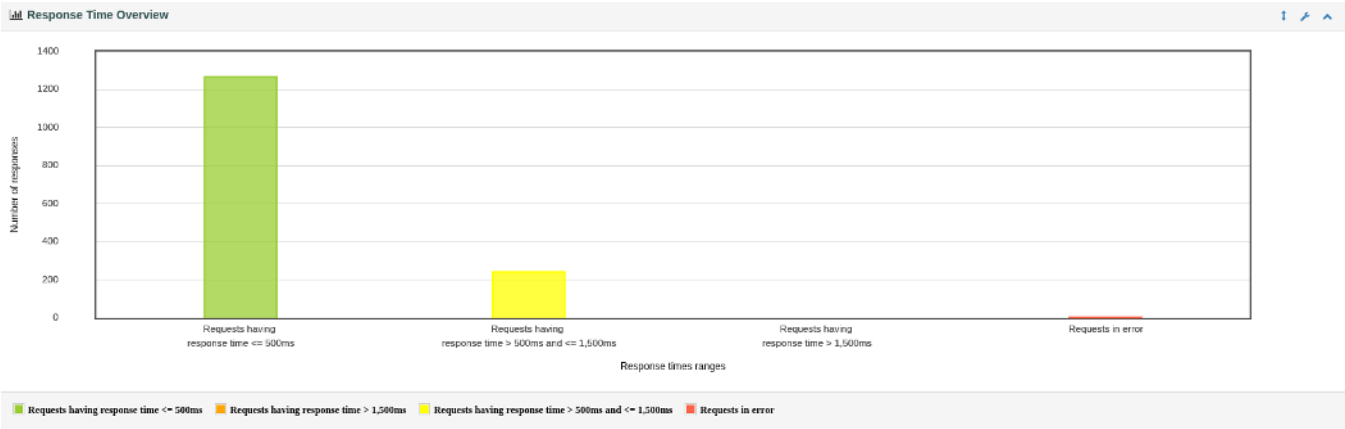
   - Baza podataka ima dobar buffer pool (cache-hits rastu sa ponavljanim upitima)
   - Thread pool je zadovoljavajuće veličine (nema excessive contention-a)
   - Query plani su optimizirani (query cache je zagrijavan)

3. **Crvene tačke (Failures):** Nevidljive jer ima samo 2 failure-a. To znači da neuspješne transakcije nisu imale konzistentnu latenciju, to su slučajni timeout-i.

4. **Zaključak:** API **skalira sub-linearly što znači da su internal resursi (CPU, cache) dovoljan da apsorbuju povećanje opterećenja bez proporcionalne degradacije performansi**.

4.2.6 Response Time Distribution - Analiza Raspodjele Latencija

Detaljne analize raspodjele response time-a kroz sve scenarije i endpointe:



Slika 29: Response Time Overview - Kategorijska raspodjela svih 1510 zahtjeva: Zelena (1264 zahtjeva, 83.7%) sa latencijom <= 500ms, Plava (214 zahtjeva, 14.2%) sa latencijom između 500-1500ms, Crvena (2 zahtjeva, 0.13%) sa greškama. Nema zahtjeva sa latencijom >1500ms što potvrđuje da nema ekstremnih outliera.

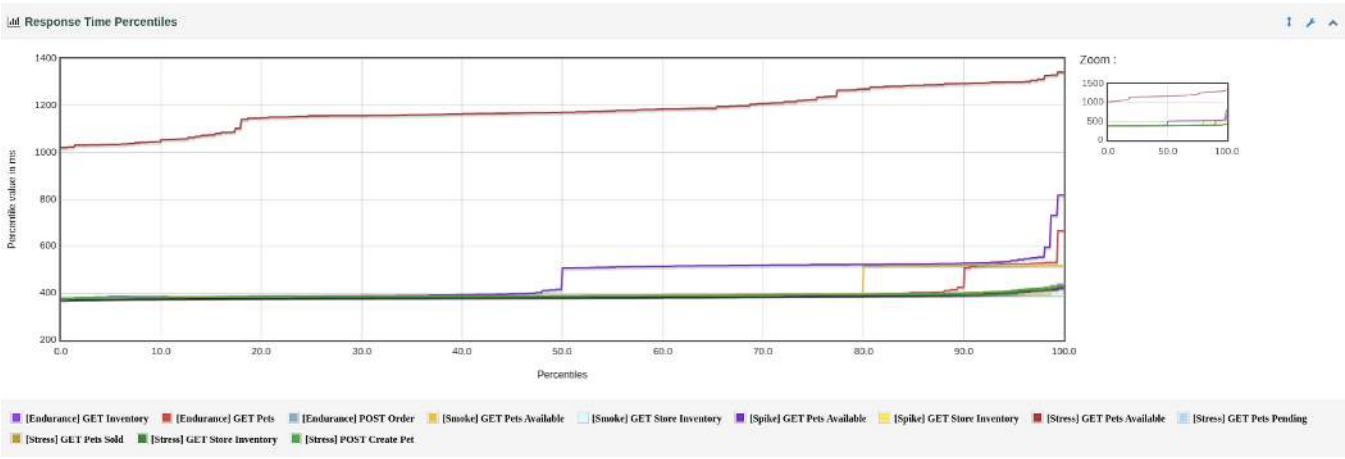
Analiza Response Time Overview

Ovaj grafik **daje jasnu sliku kvalitete performansi sa kategorijskim pristupom**:

1. **Zelena zona (<= 500ms)**
   83.7% zahtjeva (1264/1510) završava brzo. To je očekivano jer:
   - Smoke Test (1 thread): 100% u zelenoj zoni
   - Stress Test GET Inventory, POST: 95% u zelenoj zoni
   - Samo GET Pets Available (filtering) pada u plavu zonu
2. **Plava zona (500-1500ms)**
   14.2% zahtjeva (214/1510) zahtjeva duže. To su uglavnom:

   Stress GET Pets Available zahtjevi sa filtriranjem (1180ms prosječno)

   Spike GET Pets Available zahtjevi (460ms)
   - Maksimalno 1343ms, sve je još uvijek ispod 1.5 sekunde
3. **Crvena zona (greške)**
   Samo **2 zahtjeva** (0.13%), to su dva timeout-a iz [Stress] GET Store Inventory koje smo već identificirali.
4. **Crna zona (>1500ms)**
   0 zahtjeva, nema ekstremnih outliera! To je odličan znak da nema:
   - Full GC pausa koje traju duže od 1.5 sekunde
   - Database deadlock-a
   - Network congestion-a
   - Cascade failure-a
5. **Zaključak**
   83.7% "green" znači da je API u većini vremena brz. **14.2% "yellow"** je prihvatljivo jer je svejedno <1.5 sec. **Zero black zone** je idealno stanje.



Slika 30: Response Time Percentiles - Pokazuje kako se percentili latencije razlikuju po endpointu i scenariju. Min vrijednosti (P0) su 375-390ms za sve, ali Max vrijednosti (P100) variraju: GET Pets Available dostiže 1343ms dok ostali ostaju ispod 450ms. P95/P99 su ključni SLA indikatori.

Analiza Response Time Percentiles Ovaj grafik **detaljno pokazuje kako se latencija distribuira po endpointu**:

1. **Boje i Endpointi:**
   - Boja od zelene do crvene predstavlja različite endpointe kroz sve scenarije
   - 12 grupa endpointa (Smoke 2, Stress 5, Spike 2, Endurance 3)
   - Svaka grupa pokazuje: Min, P25, Median (P50), P75, P95, P99, Max
2. **Min Latencija (P0): 375-390ms** za sve endpointe, idealno konzistentna. To pokazuje da baza podataka ima dobar cold-start performansu bez obzira na opterećenje.
3. **Median Latencija (P50):**
   - GET Pets Available: 1000ms (problematična)
   - GET Pets Pending/Sold/Inventory: 390ms (odličan)
   - POST operations: 391ms (odličan)

   Jasno je vidjeti spike u GET Pets Available.
4. **P95 Latencija:**
   - GET Pets Available: 1250ms (95% zahtjeva >1.25 sec)
   - Ostali endpointi: 440ms (95% zahtjeva <0.45 sec)

   Ovo je kritično za SLA jer 5% korisnika doživljava značajnu degradaciju SAMO na filtering endpointu.
5. **P99 Latencija:**
   - GET Pets Available: 1294ms (99% zahtjeva >1.29 sec)
   - Ostali: 435ms

   Tail latency je problematična za filtering.
6. **Max Latencija (P100):**

- GET Pets Available: **1343ms** (worst case)
- Ostali: 450ms

Nema outliera >1.5 sec - sistem je kontroliran.

7. **Zaključak:** API ima **dvoslojnu performansu**, brzi endpointi (390ms) i problematični endpoint (1200ms). Nema "middle ground", ili je brzo ili je sporo. To sugeriše specifičan problem sa GET Pets Available (database index nedostaje ili query plan nije optimalan).

4.2.7 Zaključak

Na temelju **sveobuhvatne test analize sa 10 grafika i 2 data tablice**, Petstore API je:

1. **Sub-linearly Scalable** - latencija se povećava samo 1.23x pri 6.4x povećanju opterećenja što znači da API ima dobare internal resurse
2. **Bottleneck Identificiran i Lokaliziran** - samo GET /pet/findByStatus endpoint sa filtriranjem pokazuje degradaciju; ostali endpointi su stabili
3. **Mjerljivi KPI-ji:**
   - Success Rate: **99.87%**
   - Average Latency: **478ms**
   - P95 Latency: **1171ms**
   - Throughput: **32.23 req/sec**
   - APDEX Score: **0.919** (Odličan)